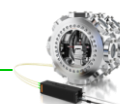




Interface Manual

AMC110 & AMC300



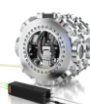
Manual version: 1.5.1
Manual modified: 08.2026

Firmware version: 1.5.1
API version: 1
attoDiscovery 1.5.1

© 2001-2026 attocube systems GmbH. Product and company names listed are trademarks or trade names of their respective companies. Any rights not expressly granted herein are reserved. ATTENTION: Specifications and technical data are subject to change without notice.

Table of Contents

1.	System Integration	4
1.1.	Connecting to Third Party Hardware	4
1.1.1.	Cabling Restrictions.....	4
1.1.2.	Pin Assignments	4
2.	Software communication & interfaces	5
2.1.	Introduction	5
2.1.1.	Web Interface	5
2.1.2.	JSON-RPC	5
2.1.3.	C/C# DLLs	5
2.1.4.	LabVIEW VIs	5
2.1.5.	Matlab library	5
2.1.6.	Python library.....	5
2.2.	Overview and implementation of the APIs.....	6
2.2.1.	JSON-RPC (JRPC2.0).....	6
2.2.2.	C Library	7
2.2.3.	C# Library	9
2.2.4.	LabVIEW	9
2.2.5.	Matlab	10
2.2.6.	Python	11
2.3.	Error handling	12
2.3.1.	C error handling	12
2.3.2.	C# error handling	12
2.3.3.	Python error handling.....	13
2.3.4.	LabVIEW error handling	13
2.4.	Available IDS API-Functions in AMC/IDS Closed Loop Mode	14
2.4.1.	Introduction	14
2.4.2.	Available IDS API-Functions	14
3.	Description of API-Functions.....	17
3.1.	About	17
3.2.	Access	19
3.3.	Amcids	23
3.4.	Control	37
3.5.	Description.....	94
3.6.	Diagnostic	98
3.7.	Move	102
3.8.	Network	126
3.9.	Res.....	152
3.10.	Rotcomp	163
3.11.	Rtin	169
3.12.	Rtout.....	187
3.13.	Status.....	203
3.14.	System_service.....	216
3.15.	Update.....	231



1. System Integration

1.1. Connecting to Third Party Hardware

The AMC110 and the AMC300 can be integrated with external systems or devices by

- combining it with third party hardware
- establishing incoming and outgoing trigger connections (/IO upgrade required)
- controlling it with individual software interfaces



CAUTION

General hazard!

Inadequate hardware connections may cause injury and are likely to damage the device or interfere with an appropriate functioning.

- Always contact attocube for technical support, before combining the device with third party hardware.
- Do carefully observe the information in this section when combining the device with third party hardware.



NOTE

attocube is not liable for any damages resulting from an unauthorized combination of the device with third party hardware. Unauthorized combination with third party hardware is not covered by attocube's warranty.

1.1.1. Cabling Restrictions

For optimal performance, obey the following combination restrictions:

- Do not to connect cabling with a wire resistance $> 5 \Omega$.
- Use EMV housings as enclosure for the D-sub connectors.
- Use extra shielded twisted pair wires for the piezo voltage supply.
- Do not connect any cable $> 5 \text{ m}$.

1.1.2. Pin Assignments

The pins of the device's positioner control cables can be found in the AMC110 or AMC300 User Manual.

2. Software communication & interfaces

You can integrate your attocube device into complex automated processes via individual software interfaces. attocube provides APIs for the programming languages C, C#, LabVIEW, Python & Matlab as well as short programming examples to get you started.

The following sections provide information on methods, commands and parameters to be used for calling up device functions with the respective language.

2.1. Introduction

The device provides a set of software communication interfaces offering a broad set of functions and options. These can be used to configure the device as well as to read out data. The following software communication interfaces exist:

- Web interface (please also refer to the User Manual)
- JSON-RPC
- C DLL
- C# DLL
- LabVIEW VIs
- Matlab library
- Python library

Most of the different functions are accessible within every interface. This is why we sort by functions not by interfaces. For every function, we show how the implementation in the particular interface is done. Anyway, first we give a short explanation of the different interfaces:



NOTE

Part of the conventions mentioned below are specific for the handling of attocube devices and are not necessarily applicable in other contexts.

2.1.1. Web Interface

The device runs a built-in webserver. This means that a web interface can be accessed via a common web browser. How to set up the IP and a first connection is given in the User Manual. The web interface is the most straightforward way to communicate with the device and almost full functionality is implemented.

2.1.2. JSON-RPC

The device allows platform-independent communication using JSON-RPC via TCP/IP and websocket. The JSON commands are the lower level that all other wrappers (e.g. Python or C) use.

2.1.3. C/C# DLLs

Based on the JSON interface, C/C# libraries are available to implement the functions within C/C#-based coding environments.

2.1.4. LabVIEW VIs

We offer ready-made VIs to have fast and easy implementation in National Instrument's LabVIEW environment.

2.1.5. Matlab library

Based on the JSON interface, we offer ready-made Matlab functions to have a fast and easy implementation in Mathwork's Matlab environment.

2.1.6. Python library

Based on the JSON interface, we offer ready-made Python functions to have a fast and easy implementation in Python environment.



2.2. Overview and implementation of the APIs

2.2.1. JSON-RPC (JRPC2.0)

Your attocube device allows platform-independent communication using JSON-RPC via TCP/IP. When using JSON-RPC, the following conventions apply.

Transport protocols

TCP

Uses communication port 9090.

Calling a JSON RPC 2.0 method

A JSON RPC method is called by sending a message to the device.

```
{ "jsonrpc": "2.0", "method": "<method>", "params": [<param [0]>, <param [1]>, ...],
  "id": <call id>, "api": <api version>}
```

<method>: String defined in chapter 2.2.

<param x>: Parameter for the method call. If PARAM is put between two “, it is a string. Without “ it is a number

<call id>: A unique id to find the corresponding answer

<api version>: A version identifier for backward compatibility, please set to 2

Example

Example:

```
{ "jsonrpc": "2.0", "method": "com.attocube.sen.displacement.getAxisDisplacement",
  "params": [1], "id": 1, "api": 2}
```

Receiving a JSON RPC 2.0 response

The JSON RPC method answer is then sent back as payload to the OK message:

```
{ "jsonrpc": "2.0", "result": [<return values [0]>, <return values [1]>, ...],
  "id": <call id>}
```

<return values [x]>:The return parameters

<call id>: The unique id of the method call

Example

Example:

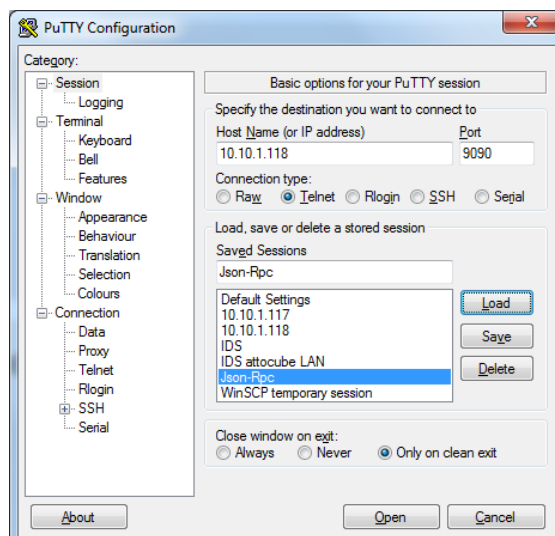
```
{ "jsonrpc": "2.0", "results": [0, 4], "id": 1}
```

Example

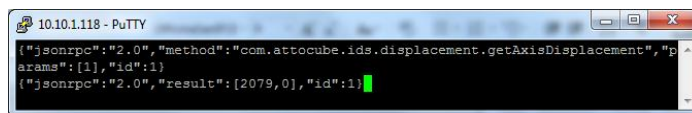
Example:

Communication via PuTTY

Open a Telnet connection with PuTTY.



Sending JSON-RPC commands in the command line interface.



```
10.10.1.118 - PuTTY
{"jsonrpc": "2.0", "method": "com.attocube.ids.displacement.getAxisDisplacement", "p
arams": [1], "id": 1}
{"jsonrpc": "2.0", "result": [2079, 0], "id": 1}
```

2.2.2. C Library

The C API is provided to integrate the device with all its functionality within your C programs.

Overview

The C API contains the following files:

Standard C API:

- libattoAMC-1.5.0.dll (dynamic link library for a windows environment)
- AMC-1.5.0_dll.lib (Provides symbol information required to link against the DLL at compile time, if desired.)
- libattoAMC-1.5.0.lib (static library for a windows environment)
- libattoAMC-1.5.0.so (dynamic library for a Linux environment)
- libattoAMC-1.5.0.a (static library for a Linux environment)
- attocubeJSONCall.h (header file for the underlying communication protocol)
- attoAMC-1.5.0.h (header file for the device specific functions)
- attoAMC.h (header file for api-version-management, to be included in your application)

Discovery C API:

- attocube-discovery-dll.dll
- attocube discovery-dll.lib
- attocube-discovery.h (header file)

Using the .dll's with different systems

Note that if you want to use the .dlls within x64 based systems outside the framework of Visual C, you might need to convert the library into a static .a format.

Establishing a connection

To connect to a device, please use (part of attocubeJSONCall.h):

int ATTOCUBE_API Connect(const char *deviceAddress, int* deviceHandle)

The device handle is the reference to the connection and the device and is input to all other device functions that are following.

To close the connection, please use:

int ATTOCUBE_API Disconnect(int * deviceHandle)

Both functions are included in the API.

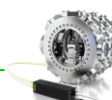
For a TCP/IP connection, the port 9090 is used.

Discovering devices within the same network

The discovery function can be used:

It searches your network for available attocube devices and returns a list of properties. This is done by a SSDP broadcast. If no devices are found, please check the device connection via TCP/IP (e.g. via the webserver). The device must be in the same subnet than the requesting PC.

IMPORTANT NOTE: These functions are part of an additional discovery .dll of the "attocube-discovery-dll" package, which is also part of the standard delivery content.



Therefore, following functions are available:

`int DLL_EXP AD_GetDeviceInfo(int index, DeviceInfo* info)`

(Get informations about a discovered device)

`void DLL_EXP AD_ReleaseInfo();`

(Release memory allocated by AD_Check)

`int DLL_EXP AD_Check(deviceType)`

(Checks discoverable devices on the network and retrieves informations)

Special data types:

`typedef struct {`

`char ipAddress[32];           /**< IP address of the device       */`

`char modelName[32];       /**< Type of the device       */`

`char serialNumber[32];   /**< Serial number of the device   */`

`char deviceName[32];   /**< Friendly name assigned to the device */`

`char macAddress[32];   /**< MAC address of the device   */`

`bool locked;           /**< Device locked by other client   */`

`} DeviceInfo;`

`typedef enum {`

`IDS,`

`AMC,`

`ATTODRY`

`} deviceType;`

2.2.3. C# Library

The C# API is provided to integrate the device with all its functionality within your C# programs.

Overview

The C# API contains of following files:

- CSharpAPIDLL.dll (compiled as “any” version)
- Newtonsoft.Json.dll (compiled as “any” version)

Establishing a connection

To connect to a device, please create an device object

```
public static Attocube<Device> client = new Attocube<Device>()
```

where <Device> is e.g. AMC or IDS

The connect function is a property of the Attocube<Device> class.

```
public void Connect(string ipAddress, int port)
```

The device handle is the reference to the connection to the device and is input to all other device functions that are following.

To close the connection, please use:

```
public void Disconnect()
```

Both functions are included in the API and part of the device class (so initialize a member of the class first).

For a TCP/IP connection, use the port 9090.

Discovering devices within the same network

The discovery function can be used:

It searches your network for available devices and returns a list of properties. This is done by an SSDP broadcast. If no devices are found, please check the device connection via TCP/IP (e.g. via the webserver) or make sure that the device is in the same subnet than your PC

Therefore, following function is available:

```
public DiscoveryData[] Check()
```

Special data type:

Type: DiscoveryData

Class for handling the data of devices discovered using the discovery protocol

2.2.4. LabVIEW

The LabVIEW API is provided to integrate the device with all its functionality within your LabVIEW VIs.

Overview

The LabVIEW API contains a LabVIEW project which contains all single function VIs and a master example VI that uses almost all functions available and that mimics the web interfaces UI for easy navigation.

Implementation

To reduce complexity and external dependencies all TCP/IP calls have been implemented with native LabVIEW TCP/IP elements. For older LabVIEW Versions where there is no native TCP/IP support, DLL based VIs have been created taking care of the TCP/IP communication.

The folders “DLLHandler” or “TCPHandler” contain the respective SubVIs handling the messaging and communication with your attocube device, which are used within all low-level VIs. Those should not be modified or used directly.



High-level Wrapper VIs

For most functions that do have both a set and a get method a higher level “controlMethod” VI has been created to reduce the number of VIs and also be as backwards compatible as possible to the older motion controller series ECC100 and ANC350. Some additional high level VIs like the deviceInfo VI have been created where multiple low-level VIs are combined into one VI and all In- and Outputs are bundled into clusters.

In case you still want to use those low-level VIs instead, they can be found inside folders that contain the word “SubVIs”. For code cleanliness it is not recommended to use those. However to keep the documentation consistent over all programming languages, only the low-level methods are documented (see chapter 3).

Establishing a connection

To connect to an device, please use the connect VI

The output is the reference to the connection to the device and is needed as an input to all other device functions that are following.

To close the connection, please use the Close VI

Both VIs are included in the API.

Discovering devices within the same network

The discovery function can be used:

It searches your network for available devices and returns a list of properties. This is done by an SSDP broadcast. If no devices are found, please check the device connection via TCP/IP (e.g. via the webserver) or make sure that your device is in the same subnet than your PC.

Therefore, the “Check.vi” is available.

2.2.5. Matlab

The Matlab API is provided to integrate the device with all its functionality within your Matlab scripts.

Establishing a connection

To connect to an device, please use:

```
[success, DeviceHandle] = connect(IPAddress, port)
```

The device handle is the reference to the connection to the device and is input to all other device functions that are following.

To close the connection, please use:

```
[success] = disconnect(DeviceHandle)
```

Both functions are included in the API.

For a TCP/IP connection, use the port 9090

2.2.6. Python

The Python API is provided to integrate the device with all its functionality within your Python programs.

Overview

The Python API contains a folder with domain specific files.

To have access to the python functions, please import the Device within your python script:

```
import <Device>
```

where <Device> is your DeviceType, e.g. AMC or IDS

Establishing a connection

To connect to an device, please use:

```
device = <Device>.Device(ipAdress)
```

```
device.connect()
```

The device handle is the reference to the connection to the device and is input to all other device functions that are following.

To close the connection, please use:

```
device.close()
```

Both functions are included in the API and part of the device class (so initialize a member of the class first).

For a TCP/IP connection, the port 9090 is used per default.

Discovering devices within the same network

The discovery function can be used:

It searches your network for available devices and returns a list of properties. This is done by an SSDP broadcast. If no devices are found, please check the device connection via TCP/IP (e.g. via the webserver) or make sure that the device is in the same subnet than your PC

Therefore, following module function is available:

```
<Device>.discover()
```

This returns a dictionary containing all found devices combined with their device information.



2.3. Error handling

2.3.1. C error handling

Introduction

The error handling in C is realized with return values, directly returned by each function. On success, the function yields zero. Negative error numbers indicate an error within the DLL itself and are specified in the header File (attocubeJSONCall.h). Positive Error numbers indicate an error in the Device, and can be translated to readable strings with `system_errorNumberToString()`

Example

```
int value;

int ret = Device_Function(device, &value);

if( ret == ATTOCUBE_Ok) {
    //success
}

else if (res < ATTOCUBE_Ok) {
    //DLL Error, e.g. not connected
}

else if (res > ATTOCUBE_Ok) {
    //Device Error
    char errorNameBuf[BUF_SIZE];
    system_errorNumberToString(device, 0, ret, errorNameBuf, BUF_SIZE);
    printf("%s", errorNameBuf)
}
```

2.3.2. C# error handling

Introduction

The error handling in C# is realized with exceptions not by error numbers. Errors can be caught using a try-catch statement. To include the device specific exceptions, the catch clause will need the `AttocubeAPIException` as argument. An example code is shown below.

Example

`public class AttocubeApiException:`

ApplicationException

Example: Exception handling

```
try
{
    attoDevice.<Method>();
}
catch (AttocubeApiException e)
{
    int err = e.ErrorCode; // passes the errorcode of type int to "err"
    string errmsg = attoDevice .ErrorNumberToString(0, err);
    // converts "err" into the corresponding error message and passes it to "errmsg" of
    type string
}
```

2.3.3. Python error handling

Introduction

The error handling in Python is realized with exceptions not by error numbers. Errors can be caught using a try-except statement. To include the specific exceptions, the catch clause will need the AttoException as argument. An example code is shown below.

Example

```
#example for exception handling

from ACS import AttoException

try:

    print(dev.<Method> ()) #OK

except AttoException as e:

    print(e)
```

2.3.4. LabVIEW error handling

Introduction

The error handling in LabVIEW is realized by using an error message variable which should be looped through all VIs. Therefore, every VI provides an error in and error out connector. Note that we divide the error variable in error messages and “real” errors, which are treated differently. Error messages have a positive error number value combined with the Boolean error status set on inactive (Boolean value on false – visualized by a green hook icon), whereas “real” errors have a negative error number values with an active (Boolean value on true – visualized by a red cross icon) error status. Error messages do not influence the execution of the following VIs, they are used to inform the user. “Real” errors hinder the execution of following VIs, they are meant to stop the program. Examples are shown below:

Example

Example for an error message. An indicator is used to visualize the error message in the front panel (see *Figure 1*). Facing a positive error code value, an error message is indicated. The status is still set on green meaning no real error available. Error messages are intended to inform the user or other functions about certain cases. If an error message is inputted in the following VI, this VI still is executed.

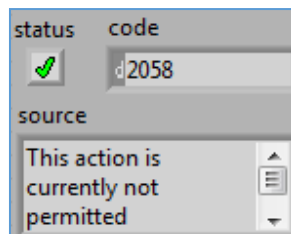


Figure 1: LabVIEW example for an error message

Example for a “real” error. Facing a negative error code value, a “real” error is indicated (see *Figure 2*). Therefore, also the status is set on red, which means that the error is active. When an active error is inputted in the next VI, this VI will not execute its function.

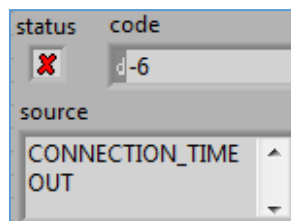


Figure 2: LabVIEW example for a “real” error



2.4. Available IDS API-Functions in AMC/IDS Closed Loop Mode

2.4.1. Introduction

When an IDS3010 is operated in AMC/IDS closed-loop mode, access to certain IDS API functions is restricted. These functions are blocked regardless of whether a full-featured or a downgraded IDS3010 is used. This restriction is necessary because certain IDS settings and parameters must remain under control of the AMC to ensure stable and reliable closed-loop operation. Modifying these parameters externally could interfere with or disable proper closed-loop functionality. If you attempt to use an API function that is not supported in AMC/IDS Closed Loop Mode, the following error message is displayed: *"This function is not available in the AMC-IDS CL configuration. For more information, please refer to the Interface Manual."* All available API functions are listed in the table in Section 2.4.2. For a detailed description of each IDS API function, please refer to the IDS Interface Manual.

2.4.2. Available IDS API-Functions

Domain	API-Function
About	getInstalledPackages
	getPackageLicence
Access	getLockStatus
	grantAccess
	lock
	unlock
Adjustment	getAdjustmentEnabled
	getContrastInPer mille
Axis	apply
	discard
	getAutoMasterAxis
	getMasterAxis
	getPassMode
	getSupportedAxes
Displacement	getAbsolutePosition
	getAbsolutePositions
	getAverageN
	getAxesDisplacement
	getAxisDisplacement
	getAxisSignalQuality
	getMeasurementEnabled
	getReferencePosition
	getReferencePositions
	setAverageN
ECU	disable
	enable
	getConnected
	getEnabled
	getHumidityInPercent

	getPressureInHPa
	getRefractiveIndex
	getRefractiveIndexCompensationMode
	getRefractiveIndexForCompensation
	getTemperatureInDegrees
	setRefractiveIndexCompensationMode
Manual	getHumidityInPercent
	getPressureInHPa
	getRefractiveIndex
	getTemperatureInDegrees
	setHumidityInPercent
	setPressureInHPa
	setRefractiveIndex
	setTemperatureInDegrees
Nlc	analyzeNonlinearities
	clearAll
	createLut
	getAnalysisStatus
	getLutApplied
	getLutDuringInit
	getLutInPm
	getLutStatus
	getNonlinearityErrorCurveInPm
	getNonlinearityErrorCurvesInPm
	getNonlinearityErrorListInPm
	setLutApplied
	setLutDuringInit
	setLutInPm
Pilotlaser	disable
	enable
	getEnabled
Realtime	enableRtAxis
	getAafAttenuation
	getAafEnabled
	getAafWindow
	getInvertedErrorSignal
	getRtAxisEnabled
	setInvertedErrorSignal
System	getAutoRestart
	getCurrentMode



	getDeviceType
	getErrorParameterSet
	getFeaturesName
	getFpgaVersion
	getInitMode
	getNbrFeaturesActivated
	getOptimizeParametersDuringInit
	getSystemError
	resetAxes
	resetAxis
	resetError
	restartMeasurement
	setAutoRestart
	setErrorParameterSet
	setOptimizeParametersDuringInit
	startMeasurement
	startOpticsAlignment
	stopMeasurement
	stopOpticsAlignment
System_service	errorNumberToRecommendation
	errorNumberToString
	factoryReset
	getDeviceName
	getFirmwareVersion
	getFluxCode
	getHardwareInfo
	getHostname
	getMacAddress
	getSerialNumber
	rebootSystem
	setDeviceName
	setTime
	softReset
	updateTimeFromInternet
Update	getLicenseUpdateProgress
	getSwUpdateProgress
	licenseUpdateBase64
	softwareUpdateBase64
	uploadLicenseBase64
	uploadSoftwareImageBase64

3. Description of API-Functions

3.1. About

getInstalledPackages

Gets list of packages installed on the device.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	value_string1	string comma separated list of packages
JSON Method		
method: com.attocube.system.about.getInstalledPackages		
params: []		
Result: [errNo, value_string1]		
C-DLL call		
int AMC_about_getInstalledPackages (int deviceHandle, char* value_string1, int size0)		
Python		
value_string1 = [dev].about.getInstalledPackages()		
Matlab		
[value_string1] = about_getInstalledPackages ()		
C#		
string value = [Device]. About_GetInstalledPackages ()		
LabVIEW		
getInstalledPackages.vi		



getPackageLicense

Gets the license for a specific package.

Function specific parameters		
In	pckg	package name string
Out	errNo	error code, if there was an error, otherwise 0 for ok
	value_string1	string license for this package
JSON Method		
method: com.attocube.system.about.getPackageLicense		
params: [pckg]		
Result: [errNo, value_string1]		
C-DLL call		
int AMC_about_getPackageLicense (int deviceHandle, const char* pckg, char* value_string1, int size0)		
Python		
value_string1 = [dev].about.getPackageLicense (pckg)		
Matlab		
[value_string1] = about_getPackageLicense (pckg)		
C#		
string value = [Device]. About_GetPackageLicense (string pckg)		
LabVIEW		
getPackageLicense.vi		

3.2. Access

getLockStatus

Gets the lock status of the device and if the current client is authorized to use the device.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	locked	true = locked; false = unlocked
	authorized	true if the client is authorized, else false
JSON Method		
method: getLockStatus		
params: []		
Result: [errNo, locked, authorized]		
C-DLL call		
int AMC_getLockStatus (int deviceHandle, bool* locked, bool* authorized)		
Python		
locked, authorized = [dev].access.getLockStatus()		
Matlab		
[locked, authorized] = AMC_getLockStatus ()		
C#		
Tuple<bool,bool> value = [Device]. GetLockStatus ()		
LabVIEW		
getLockStatus.vi		



grantAccess

Grants access to a locked device by checking against the password.

Function specific parameters		
In	password	current password string
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: grantAccess		
params: [password]		
Result: [errNo]		
C-DLL call		
int AMC_grantAccess (int deviceHandle, const char* password)		
Python		
[dev].access.grantAccess(password)		
Matlab		
[] = AMC_grantAccess (password)		
C#		
void value = [Device]. GrantAccess (string password)		
LabVIEW		
grantAccess.vi		

lock

Locks the device with a given password. Afterwards, the device functions are only accessible with this password. The computer IP address, which locks the device, is automatically added to the device and doesn't need the password for functions access.

Function specific parameters		
In	password	set password string
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: lock		
params: [password]		
Result: [errNo]		
C-DLL call		
int AMC_lock (int deviceHandle, const char* password)		
Python		
[dev].access.lock (password)		
Matlab		
[] = AMC_lock (password)		
C#		
void value = [Device]. Lock (string password)		
LabVIEW		
lock.vi		



unlock

Unlocks the device. Afterwards, the device functions are accessible without any password. For grant access without unlocking the device, please use the grantAccess function.

Function specific parameters		
In	password	current password string
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: unlock		
params: [password]		
Result: [errNo]		
C-DLL call		
int AMC_unlock (int deviceHandle, const char* password)		
Python		
[dev].access.unlock(password)		
Matlab		
[] = AMC_unlock (password)		
C#		
void value = [Device]. Unlock (string password)		
LabVIEW		
unlock.vi		

3.3. Amcids

getAutoStop

Reads the current setting of auto_stop in case of an error raised by the IDS.

If True, the closedloop movement is stopped automatically.

Function specific parameters		
In	axis	Axis [0 1 2] on which the autostop is read
Out	errNo	Error number if one occurred, 0 in case of no error
	auto_stop	True if closedloop move shall stop automatically on error reported by IDS
JSON Method		
method: com.attocube.amc.amcids.getAutoStop		
params: [axis]		
Result: [errNo, auto_stop]		
C-DLL call		
int AMC_amcids_getAutoStop (int deviceHandle, int axis, bool* auto_stop)		
Python		
auto_stop = [dev].amcids.getAutoStop(axis)		
Matlab		
[auto_stop] = amcids_getAutoStop(axis)		
C#		
bool value = [Device].Amcids_GetAutoStop(int axis)		
LabVIEW		
getAutoStop.vi		



getError

Reads the statuses of the RT- and ethernet-connection.

All return parameters follow the following logic:

0: no error occurred since last reset

1: error is occurring at the moment

2: error occurred in the past, but not at the moment

3: error is occurring at the moment and latched

Function specific parameters		
Out	errNo	Error number if one occurred, 0 in case of no error
	itf_ids_error_all_axes	indicates an error reported by the IDS on axis 1, 2 or 3 (see above for value-explanation)
	itf_link_error_all_axes	indicates a link error detected on axis 1, 2 or 3 (see above for value-explanation)
	itf_ids_error_ax_1	indicates an error reported by the IDS on axis 1 (e.g. beam interrupt). (see above for value-explanation)
	itf_ids_error_ax_2	indicates an error reported by the IDS on axis 2 (e.g. beam interrupt). (see above for value-explanation)
	itf_ids_error_ax_3	indicates an error reported by the IDS on axis 3 (e.g. beam interrupt). (see above for value-explanation)
	itf_link_error_ax_1	indicates a link error detected on axis 1 (e.g. wiring/cable problem). (see above for value-explanation)
	itf_link_error_ax_2	indicates a link error detected on axis 2 (e.g. wiring/cable problem). (see above for value-explanation)
	itf_link_error_ax_3	indicates a link error detected on axis 3 (e.g. wiring/cable problem). (see above for value-explanation)
	eth_con_error	indicates an ethernet connection error between AMC and IDS (see above for value-explanation)
JSON Method		
method: com.attocube.amc.amcids.getError		
params: []		
Result: [errNo, itf_ids_error_all_axes, itf_link_error_all_axes, itf_ids_error_ax_1, itf_ids_error_ax_2, itf_ids_error_ax_3, itf_link_error_ax_1, itf_link_error_ax_2, itf_link_error_ax_3, eth_con_error]		
C-DLL call		
int AMC_amcids_getError (int deviceHandle, int* itf_ids_error_all_axes, int* itf_link_error_all_axes, int* itf_ids_error_ax_1, int* itf_ids_error_ax_2, int* itf_ids_error_ax_3, int* itf_link_error_ax_1, int* itf_link_error_ax_2, int* itf_link_error_ax_3, int* eth_con_error)		
Python		
itf_ids_error_all_axes, itf_link_error_all_axes, itf_ids_error_ax_1, itf_ids_error_ax_2, itf_ids_error_ax_3, itf_link_error_ax_1, itf_link_error_ax_2, itf_link_error_ax_3, eth_con_error = [dev].amcids.getError()		
Matlab		
[itf_ids_error_all_axes, itf_link_error_all_axes, itf_ids_error_ax_1, itf_ids_error_ax_2, itf_ids_error_ax_3, itf_link_error_ax_1, itf_link_error_ax_2, itf_link_error_ax_3, eth_con_error] = amcids_getError()		
C#		
Tuple<int,int,int,int,int,int, Tuple<int,int,int>> value = [Device].Amcids_GetError()		
LabVIEW		
getError.vi		

getLowerSoftLimit

Gets the lower boundary of the soft limit protection.

This protection is needed if the IDS working range is smaller than the positioners travel range.

It is no hard limit, so, it is possible to overshoot it!

Function specific parameters		
In	axis	Axis [0 1 2] of the AMC to get the soft limit status from
Out	errNo	Error number if one occurred, 0 in case of no error
	limit	Lower boundary in micrometers
JSON Method		
method: com.attocube.amc.amcids.getLowerSoftLimit		
params: [axis]		
Result: [errNo, limit]		
C-DLL call		
int AMC_amcids_getLowerSoftLimit (int deviceHandle, int axis, double* limit)		
Python		
limit = [dev].amcids.getLowerSoftLimit(axis)		
Matlab		
[limit] = amcids_getLowerSoftLimit(axis)		
C#		
double value = [Device].Amcids_GetLowerSoftLimit(int axis)		
LabVIEW		
getLowerSoftLimit.vi		



getSoftLimitEnabled

Gets whether the soft limit protection is enabled.

This protection is needed if the IDS working range is smaller than the positioners travel range.

It is no hard limit, so, it is possible to overshoot it!

Function specific parameters		
In	axis	Axis [0 1 2] of the AMC to get the soft limit status from
Out	errNo	Error number if one occurred, 0 in case of no error
	enabled	True, if the soft limit should be enabled on this axis
JSON Method		
method: com.attocube.amc.amcids.getSoftLimitEnabled		
params: [axis]		
Result: [errNo, enabled]		
C-DLL call		
int AMC_amcids_getSoftLimitEnabled (int deviceHandle, int axis, bool* enabled)		
Python		
enabled = [dev].amcids.getSoftLimitEnabled(axis)		
Matlab		
[enabled] = amcids_getSoftLimitEnabled (axis)		
C#		
bool value = [Device]. Amcids_GetSoftLimitEnabled (int axis)		
LabVIEW		
getSoftLimitEnabled.vi		

getSoftLimitRange

Reads the lower and upper boundary of the soft limit protection.

Function specific parameters		
In	axis	Axis of the AMC to get the soft limit status from
Out	errNo	Error number, is converted into an exception automatically
	lower_limit	Lower boundary in micrometers
	upper_limit	Upper boundary in micrometers
JSON Method		
method: com.attocube.amc.amcids.getSoftLimitRange		
params: [axis]		
Result: [errNo, lower_limit, upper_limit]		
C-DLL call		
int AMC_amcids_getSoftLimitRange (int deviceHandle, int axis, int* lower_limit, int* upper_limit)		
Python		
lower_limit, upper_limit = [dev].amcids.getSoftLimitRange(axis)		
Matlab		
[lower_limit, upper_limit] = amcids_getSoftLimitRange (axis)		
C#		
Tuple<int,int> value = [Device].Amcids_GetSoftLimitRange(int axis)		
LabVIEW		
getSoftLimitRange.vi		



getSoftLimitReached

Gets whether the current position is out of the soft limit boundaries.

This protection is needed if the IDS working range is smaller than the positioners travel range.

It is no hard limit, so, it is possible to overshoot it!

Function specific parameters		
In	axis	Axis [0 1 2] of the AMC to get the soft limit status from
Out	errNo	Error number if one occurred, 0 in case of no error
	enabled	True, if the position is not within the boundaries
JSON Method		
method: com.attocube.amc.amcids.getSoftLimitReached		
params: [axis]		
Result: [errNo, enabled]		
C-DLL call		
int AMC_amcids_getSoftLimitReached (int deviceHandle, int axis, bool* enabled)		
Python		
enabled = [dev].amcids.getSoftLimitReached(axis)		
Matlab		
[enabled] = amcids_getSoftLimitReached (axis)		
C#		
bool value = [Device]. Amcids_GetSoftLimitReached (int axis)		
LabVIEW		
getSoftLimitReached.vi		

getUpperSoftLimit

Gets the upper boundary of the soft limit protection.

This protection is needed if the IDS working range is smaller than the positioners travel range.

It is no hard limit, so, it is possible to overshoot it!

Function specific parameters		
In	axis	Axis [0 1 2] of the AMC to get the soft limit status from
Out	errNo	Error number if one occurred, 0 in case of no error
	limit	Upper boundary in micrometers
JSON Method		
method: com.attocube.amc.amcids.getUpperSoftLimit		
params: [axis]		
Result: [errNo, limit]		
C-DLL call		
int AMC_amcids_getUpperSoftLimit (int deviceHandle, int axis, double* limit)		
Python		
limit = [dev].amcids.getUpperSoftLimit(axis)		
Matlab		
[limit] = amcids_getUpperSoftLimit(axis)		
C#		
double value = [Device].Amcids_GetUpperSoftLimit(int axis)		
LabVIEW		
getUpperSoftLimit.vi		



resetError

Resets the latched error information of the RT- and ethernet-errors (see `getError-method`)

The axis-parameter only applies to the RT-errors and ids-errors - the ethernet-error is not reset axis-specifically

Function specific parameters		
In	axis	The index of the axis [0 1 2] whose RT- and IDS-error information should be reset. Note, that the IDS error is reset for all three axes, if an error is detected on the given axis.
Out	errNo	Error number if one occurred, 0 in case of no error
JSON Method		
method: com.attocube.amc.amcids.resetError		
params: [axis]		
Result: [errNo]		
C-DLL call		
int AMC_amcids_resetError (int deviceHandle, int axis)		
Python		
[dev].amcids.resetError (axis)		
Matlab		
[] = amcids_resetError (axis)		
C#		
void value = [Device]. Amcids_ResetError (int axis)		
LabVIEW		
resetError.vi		

resetIdsAxis

Resets the position value to zero of a specific measurement axis.

Use this for positioners with an IDS as sensor.

This method does not work for NUM and RES sensors. Use `com.attocube.amc.control.resetAxis` instead.

Function specific parameters		
In	axis	Axis [0 1 2] of the IDS to reset the position
Out	errNo	Error number if one occurred, 0 in case of no error
JSON Method		
method: <code>com.attocube.amc.amcids.resetIdsAxis</code>		
params: [axis]		
Result: [errNo]		
C-DLL call		
int AMC_amcids_resetIdsAxis (int deviceHandle, int axis)		
Python		
<code>[dev].amcids.resetIdsAxis(axis)</code>		
Matlab		
<code>[] = amcids_resetIdsAxis(axis)</code>		
C#		
void value = [Device]. Amcids_ResetIdsAxis (int axis)		
LabVIEW		
resetIdsAxis.vi		



setAutoStop

When `auto_stop` is set to 'True', the AMC automatically stops cl-movement in case of an error indicated by the IDS (e.g. beam-interrupt)

Note, that the AMC always stops closed-loop-movement automatically, if the transferred position is invalid (e.g. if the sensor-cable was disconnected)

Note, that in case of active 'stop at end of travel'-setting (check function `setControlEotOutputDeactive`), the movement would stop despite setting `auto_stop` to 'False' for an error reported by the IDS, because the AMC interprets the invalid

position-value as "end of travel".

With `getError` the states of the errors can be retrieved.

Function specific parameters		
In	axis	indicates the device axis [0 1 2] on which the auto-stop shall be applied
	auto_stop	if 'True' the closedloop move will stop, if the IDS reports errors on this axis
Out	errNo	Error number if one occurred, 0 in case of no error
JSON Method		
method: <code>com.attocube.amc.amcids.setAutoStop</code>		
params: <code>[axis, auto_stop]</code>		
Result: <code>[errNo]</code>		
C-DLL call		
int AMC_amcids_setAutoStop (int deviceHandle, int axis, bool auto_stop)		
Python		
<code>[dev].amcids.setAutoStop(axis, auto_stop)</code>		
Matlab		
<code>[] = amcids_setAutoStop(axis, auto_stop)</code>		
C#		
void value = <code>[Device].Amcids_SetAutoStop</code> (int axis, bool auto_stop)		
LabVIEW		
setAutoStop.vi		

setLowerSoftLimit

Sets the lower boundary of the soft limit protection.

This protection is needed if the IDS working range is smaller than the positioners travel range.

It is no hard limit, so, it is possible to overshoot it!

Function specific parameters		
In	axis	Axis [0 1 2] of the AMC where the soft limit should be changed
	limit	Lower boundary in micrometers
Out	errNo	Error number if one occurred, 0 in case of no error
JSON Method		
method: com.attocube.amc.amcids.setLowerSoftLimit		
params: [axis, limit]		
Result: [errNo]		
C-DLL call		
int AMC_amcids_setLowerSoftLimit (int deviceHandle, int axis, double limit)		
Python		
[dev].amcids.setLowerSoftLimit (axis, limit)		
Matlab		
[] = amcids_setLowerSoftLimit (axis, limit)		
C#		
void value = [Device]. Amcids_SetLowerSoftLimit (int axis, double limit)		
LabVIEW		
setLowerSoftLimit.vi		



setSoftLimitEnabled

Enables/disables the soft limit protection.

This protection is needed if the IDS working range is smaller than the positioners travel range.

It is no hard limit, so, it is possible to overshoot it!

Function specific parameters		
In	axis	Axis [0 1 2] of the AMC where the soft limit should be changed
	enabled	True, if the soft limit should be enabled on this axis
Out	errNo	Error number if one occurred, 0 in case of no error
JSON Method		
method: com.attocube.amc.amcids.setSoftLimitEnabled		
params: [axis, enabled]		
Result: [errNo]		
C-DLL call		
int AMC_amcids_setSoftLimitEnabled (int deviceHandle, int axis, bool enabled)		
Python		
[dev].amcids.setSoftLimitEnabled (axis, enabled)		
Matlab		
[] = amcids_setSoftLimitEnabled (axis, enabled)		
C#		
void value = [Device]. Amcids_SetSoftLimitEnabled (int axis, bool enabled)		
LabVIEW		
setSoftLimitEnabled.vi		

setSoftLimitRange

Sets the lower and upper boundry of the soft limit protection in micrometers.

This protection is needed if the IDS working range is smaller than the positioners travel range.

It is no hard limit, so, it is possible to overshoot it!

The given values are interpreted as absolute positions (reference position + displacement).

Function specific parameters		
In	axis	Axis [0 1 2] of the AMC where the soft limits should be changed
	lowerLimit	lower boundary in micrometers as absolute position; has to be lower than upperLimit
	upperLimit	upper boundary in micrometers as absolute position; has to be higher than lowerLimit
Out	errNo	Error number if one occurred, 0 in case of no error
JSON Method		
method: com.attocube.amc.amcids.setSoftLimitRange		
params: [axis, lowerLimit, upperLimit]		
Result: [errNo]		
C-DLL call		
int AMC_amcids_setSoftLimitRange (int deviceHandle, int axis, int lowerLimit, int upperLimit)		
Python		
[dev].amcids.setSoftLimitRange (axis, lowerLimit, upperLimit)		
Matlab		
[] = amcids_setSoftLimitRange (axis, lowerLimit, upperLimit)		
C#		
void value = [Device]. Amcids_SetSoftLimitRange (int axis, int lowerLimit, int upperLimit)		
LabVIEW		
setSoftLimitRange.vi		



setUpperSoftLimit

Sets the upper boundary of the soft limit protection.

This protection is needed if the IDS working range is smaller than the positioners travel range.

It is no hard limit, so, it is possible to overshoot it!

Function specific parameters		
In	axis	Axis [0 1 2] of the AMC where the soft limit should be changed
	limit	Upper boundary in micrometers
Out	errNo	Error number if one occurred, 0 in case of no error
JSON Method		
method: com.attocube.amc.amcids.setUpperSoftLimit		
params: [axis, limit]		
Result: [errNo]		
C-DLL call		
int AMC_amcids_setUpperSoftLimit (int deviceHandle, int axis, double limit)		
Python		
[dev].amcids.setUpperSoftLimit (axis, limit)		
Matlab		
[] = amcids_setUpperSoftLimit (axis, limit)		
C#		
void value = [Device]. Amcids_SetUpperSoftLimit (int axis, double limit)		
LabVIEW		
setUpperSoftLimit.vi		

3.4. Control

MultiAxisPositioning

By means of this function you can set target positions for all axes simultaneously (depending on the boolean parameters set1, set2, set3)

Additionally the current position, the status of the reference and the reference position of all three axes are returned.

This function can only be used in conjunction with NUM.

Function specific parameters		
In	set1	set target position on axis1, if "false" target1-parameter is ignored
	set2	set target position on axis2, if "false" target2-parameter is ignored
	set3	set target position on axis3, if "false" target3-parameter is ignored
	target1	target position of axis 1
	target2	target position of axis 2
	target3	target position of axis 3
Out	errNo	errNo
	ref1	Status of axis 1
	ref2	Status of axis 2
	ref3	Status of axis 3
	refpos1	reference Position of axis 1
	refpos2	reference Position of axis 2
	refpos3	reference Position of axis 3
	pos1	position of axis 1
	pos2	position of axis 2
	pos3	position of axis 3
JSON Method		
method: com.attocube.amc.control.MultiAxisPositioning		
params: [set1, set2, set3, target1, target2, target3]		
Result: [errNo, ref1, ref2, ref3, refpos1, refpos2, refpos3, pos1, pos2, pos3]		
C-DLL call		
int AMC_control_MultiAxisPositioning (int deviceHandle, bool set1, bool set2, bool set3, int target1, int target2, int target3, bool* ref1, bool* ref2, bool* ref3, int* refpos1, int* refpos2, int* refpos3, int* pos1, int* pos2, int* pos3)		
Python		
ref1, ref2, ref3, refpos1, refpos2, refpos3, pos1, pos2, pos3 = [dev].control.MultiAxisPositioning (set1, set2, set3, target1, target2, target3)		
Matlab		
[ref1, ref2, ref3, refpos1, refpos2, refpos3, pos1, pos2, pos3] = control_MultiAxisPositioning (set1, set2, set3, target1, target2, target3)		
C#		
Tuple<bool,bool,bool,int,int,int, Tuple<int,int,int>> value = [Device]. Control_MultiAxisPositioning (bool set1, bool set2, bool set3, int target1, int target2, int target3)		
LabVIEW		
MultiAxisPositioning.vi		



MultiAxisPositioningWithTime

In addition to "MultiAxisPositioning", this function returns a timestamp to the current position of each axis.

Please refer to the description of "getPositionWithTime" for more information on the timestamp.

This function can only be used in conjunction with NUM.

Function specific parameters		
In	set1	set target position on axis1, if "false" target1-parameter is ignored
	set2	set target position on axis2, if "false" target2-parameter is ignored
	set3	set target position on axis3, if "false" target3-parameter is ignored
	target1	target position of axis 1
	target2	target position of axis 2
	target3	target position of axis 3
Out	errNo	errNo
	ref1	Status of axis 1
	ref2	Status of axis 2
	ref3	Status of axis 3
	refpos1	reference Position of axis 1
	refpos2	reference Position of axis 2
	refpos3	reference Position of axis 3
	pos1	position of axis 1
	pos2	position of axis 2
	pos3	position of axis 3
	time1	timestamp of axis 1
	time2	timestamp of axis 2
	time3	timestamp of axis 3
JSON Method		
method: com.attocube.amc.control.MultiAxisPositioningWithTime		
params: [set1, set2, set3, target1, target2, target3]		
Result: [errNo, ref1, ref2, ref3, refpos1, refpos2, refpos3, pos1, pos2, pos3, time1, time2, time3]		
C-DLL call		
int AMC_control_MultiAxisPositioningWithTime (int deviceHandle, bool set1, bool set2, bool set3, int target1, int target2, int target3, bool* ref1, bool* ref2, bool* ref3, int* refpos1, int* refpos2, int* refpos3, int* pos1, int* pos2, int* pos3, double* time1, double* time2, double* time3)		
Python		
ref1, ref2, ref3, refpos1, refpos2, refpos3, pos1, pos2, pos3, time1, time2, time3 = [dev].control.MultiAxisPositioningWithTime (set1, set2, set3, target1, target2, target3)		
Matlab		
[ref1, ref2, ref3, refpos1, refpos2, refpos3, pos1, pos2, pos3, time1, time2, time3] = control_MultiAxisPositioningWithTime (set1, set2, set3, target1, target2, target3)		
C#		
Tuple<bool,bool,bool,int,int,int, Tuple<int,int,int,double,double,double>> value = [Device].Control_MultiAxisPositioningWithTime (bool set1, bool set2, bool set3, int target1, int target2, int target3)		
LabVIEW		
MultiAxisPositioningWithTime.vi		

getActorName

This function gets the name of the positioner of the selected axis.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	actor_name	actor_name
JSON Method		
method: com.attocube.amc.control.getActorName		
params: [axis]		
Result: [errNo, actor_name]		
C-DLL call		
int AMC_control_getActorName (int deviceHandle, int axis, char* actor_name, int size0)		
Python		
actor_name = [dev].control.getActorName(axis)		
Matlab		
[actor_name] = control_getActorName (axis)		
C#		
string value = [Device]. Control_GetActorName (int axis)		
LabVIEW		
getActorName.vi		



getActorParametersActorName

Control the actors parameter: actor name

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	actorname	actorname
JSON Method		
method: com.attocube.amc.control.getActorParametersActorName		
params: [axis]		
Result: [errNo, actorname]		
C-DLL call		
int AMC_control_getActorParametersActorName (int deviceHandle, int axis, char* actorname, int size0)		
Python		
actorname = [dev].control.getActorParametersActorName(axis)		
Matlab		
[actorname] = control_getActorParametersActorName (axis)		
C#		
string value = [Device]. Control_GetActorParametersActorName (int axis)		
LabVIEW		
getActorParametersActorName.vi		

getActorParametersJson

This function reads the current values of the given set of actor-parameters and returns them as key-value-pairs in a json-string

Function specific parameters		
In	axis	[0 1 2]
	param_list	comma separated list of parameters to be read out
Out	errNo	errorCode
	parameters	json_dict of requested parameters
JSON Method		
method: com.attocube.amc.control.getActorParametersJson		
params: [axis, param_list]		
Result: [errNo, parameters]		
C-DLL call		
int AMC_control_getActorParametersJson (int deviceHandle, int axis, const char* param_list, char* parameters, int size0)		
Python		
parameters = [dev].control.getActorParametersJson(axis, param_list)		
Matlab		
[parameters] = control_getActorParametersJson (axis, param_list)		
C#		
string value = [Device]. Control_GetActorParametersJson (int axis, string param_list)		
LabVIEW		
getActorParametersJson.vi		



getActorSensitivity

Get the setting for the actor parameter sensitivity

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	sensitivity	[0 (fast approach) .. 10 (slow approach)] The approach behaviour changes exponentially with the sensitivity
JSON Method		
method: com.attocube.amc.control.getActorSensitivity		
params: [axis]		
Result: [errNo, sensitivity]		
C-DLL call		
int AMC_control_getActorSensitivity (int deviceHandle, int axis, int* sensitivity)		
Python		
sensitivity = [dev].control.getActorSensitivity (axis)		
Matlab		
[sensitivity] = control_getActorSensitivity (axis)		
C#		
int value = [Device]. Control_GetActorSensitivity (int axis)		
LabVIEW		
getActorSensitivity.vi		

getActorType

This function gets the type of the positioner of the selected axis.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	actor_type	0: linear, 1: rotator, 2: goniometer
JSON Method		
method: com.attocube.amc.control.getActorType		
params: [axis]		
Result: [errNo, actor_type]		
C-DLL call		
int AMC_control_getActorType (int deviceHandle, int axis, int* actor_type)		
Python		
actor_type = [dev].control.getActorType(axis)		
Matlab		
[actor_type] = control_getActorType (axis)		
C#		
int value = [Device]. Control_GetActorType (int axis)		
LabVIEW		
getActorType.vi		



getAutoMeasure

This function returns if the automeasurement on axis enable is enabled

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enable	true: enable automeasurement, false: disable automeasurement
JSON Method		
method: com.attocube.amc.control.getAutoMeasure		
params: [axis]		
Result: [errNo, enable]		
C-DLL call		
int AMC_control_getAutoMeasure (int deviceHandle, int axis, bool* enable)		
Python		
enable = [dev].control.getAutoMeasure(axis)		
Matlab		
[enable] = control_getAutoMeasure (axis)		
C#		
bool value = [Device]. Control_GetAutoMeasure (int axis)		
LabVIEW		
getAutoMeasure.vi		

getAxesSensorSources

This function gets the current external sensor configuration for all axes
 It is only available when the feature AMC/IDS closed loop has been activated
 It returns an integer representing the configuration:

- 0: intern,intern,intern
- 1: extern,extern,extern
- 2: extern,extern,intern
- 3: intern,intern,extern
- 4: inconsistent setting

If this function returns '4', please run setAxesSensorSources with your desired configuration from above.

Function specific parameters		
Out	errNo	errNo
	extSensCfg	[0: intern,intern,intern 1: extern,extern,extern 2: extern,extern,intern 3: intern,intern,extern 4:inconsitent]
JSON Method		
method: com.attocube.amc.control.getAxesSensorSources		
params: []		
Result: [errNo, extSensCfg]		
C-DLL call		
int AMC_control_getAxesSensorSources (int deviceHandle, int* extSensCfg)		
Python		
extSensCfg = [dev].control.getAxesSensorSources()		
Matlab		
[extSensCfg] = control_getAxesSensorSources ()		
C#		
int value = [Device]. Control_GetAxesSensorSources ()		
LabVIEW		
getAxesSensorSources.vi		



getControlAmplitude

This function gets the amplitude of the actuator signal of the selected axis.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	amplitude	in mV
JSON Method		
method: com.attocube.amc.control.getControlAmplitude		
params: [axis]		
Result: [errNo, amplitude]		
C-DLL call		
int AMC_control_getControlAmplitude (int deviceHandle, int axis, int* amplitude)		
Python		
amplitude = [dev].control.getControlAmplitude (axis)		
Matlab		
[amplitude] = control_getControlAmplitude (axis)		
C#		
int value = [Device]. Control_GetControlAmplitude (int axis)		
LabVIEW		
getControlAmplitude.vi		

getControlAutoReset

This function resets the position every time the reference position is detected
this setting only has an effect for NUM-positioners.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enabled	boolean
JSON Method		
method: com.attocube.amc.control.getControlAutoReset		
params: [axis]		
Result: [errNo, enabled]		
C-DLL call		
int AMC_control_getControlAutoReset (int deviceHandle, int axis, bool* enabled)		
Python		
enabled = [dev].control.getControlAutoReset(axis)		
Matlab		
[enabled] = control_getControlAutoReset (axis)		
C#		
bool value = [Device]. Control_GetControlAutoReset (int axis)		
LabVIEW		
getControlAutoReset.vi		



getControlFixOutputVoltage

This function gets the DC level output of the selected axis.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	amplitude_mv	in mV
JSON Method		
method: com.attocube.amc.control.getControlFixOutputVoltage		
params: [axis]		
Result: [errNo, amplitude_mv]		
C-DLL call		
int AMC_control_getControlFixOutputVoltage (int deviceHandle, int axis, int* amplitude_mv)		
Python		
amplitude_mv = [dev].control.getControlFixOutputVoltage(axis)		
Matlab		
[amplitude_mv] = control_getControlFixOutputVoltage(axis)		
C#		
int value = [Device].Control_GetControlFixOutputVoltage(int axis)		
LabVIEW		
getControlFixOutputVoltage.vi		

getControlFrequency

This function gets the frequency of the actuator signal of the selected axis.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	frequency	in mHz
JSON Method		
method: com.attocube.amc.control.getControlFrequency		
params: [axis]		
Result: [errNo, frequency]		
C-DLL call		
int AMC_control_getControlFrequency (int deviceHandle, int axis, int* frequency)		
Python		
frequency = [dev].control.getControlFrequency (axis)		
Matlab		
[frequency] = control_getControlFrequency (axis)		
C#		
int value = [Device]. Control_GetControlFrequency (int axis)		
LabVIEW		
getControlFrequency.vi		



getControlMove

This function gets the approach of the selected axis' positioner to the target position.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enable	boolean true: closed loop control enabled, false: closed loop control disabled
JSON Method		
method: com.attocube.amc.control.getControlMove		
params: [axis]		
Result: [errNo, enable]		
C-DLL call		
int AMC_control_getControlMove (int deviceHandle, int axis, bool* enable)		
Python		
enable = [dev].control.getControlMove(axis)		
Matlab		
[enable] = control_getControlMove (axis)		
C#		
bool value = [Device]. Control_GetControlMove (int axis)		
LabVIEW		
getControlMove.vi		

getControlOutput

This function gets the activation status of the axis.

If active, move commands are accepted.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enabled	power status (true = enabled,false = disabled)
JSON Method		
method: com.attocube.amc.control.getControlOutput		
params: [axis]		
Result: [errNo, enabled]		
C-DLL call		
int AMC_control_getControlOutput (int deviceHandle, int axis, bool* enabled)		
Python		
enabled = [dev].control.getControlOutput(axis)		
Matlab		
[enabled] = control_getControlOutput (axis)		
C#		
bool value = [Device]. Control_GetControlOutput (int axis)		
LabVIEW		
getControlOutput.vi		



getControlReferenceAutoUpdate

This function gets the status of whether the reference position is updated when the reference mark is hit

When this function is disabled, the reference marking will be considered only the first time and after then ignored
this setting only has an effect for NUM-positioners.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enabled	boolean
JSON Method		
method: com.attocube.amc.control.getControlReferenceAutoUpdate		
params: [axis]		
Result: [errNo, enabled]		
C-DLL call		
int AMC_control_getControlReferenceAutoUpdate (int deviceHandle, int axis, bool* enabled)		
Python		
enabled = [dev].control.getControlReferenceAutoUpdate(axis)		
Matlab		
[enabled] = control_getControlReferenceAutoUpdate (axis)		
C#		
bool value = [Device]. Control_GetControlReferenceAutoUpdate (int axis)		
LabVIEW		
getControlReferenceAutoUpdate.vi		

getControlTargetRange

This function gets the range around the target position in which the flag "In Target Range" becomes active.

Please note, that this setting is only used to identify the target position, it has no influence on the closed loop control.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	targetrange	in nm
JSON Method		
method: com.attocube.amc.control.getControlTargetRange		
params: [axis]		
Result: [errNo, targetrange]		
C-DLL call		
int AMC_control_getControlTargetRange (int deviceHandle, int axis, int* targetrange)		
Python		
targetrange = [dev].control.getControlTargetRange(axis)		
Matlab		
[targetrange] = control_getControlTargetRange (axis)		
C#		
int value = [Device]. Control_GetControlTargetRange (int axis)		
LabVIEW		
getControlTargetRange.vi		



getCrosstalkThreshold

This function gets the threshold range and slip phase time which is used while moving another axis

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	range	in pm
	time	after slip phase which is waited until the controller is acting again in microseconds
JSON Method		
method: com.attocube.amc.control.getCrosstalkThreshold		
params: [axis]		
Result: [errNo, range, time]		
C-DLL call		
int AMC_control_getCrosstalkThreshold (int deviceHandle, int axis, int* range, int* time)		
Python		
range, time = [dev].control.getCrosstalkThreshold (axis)		
Matlab		
[range, time] = control_getCrosstalkThreshold (axis)		
C#		
Tuple<int,int> value = [Device]. Control_GetCrosstalkThreshold (int axis)		
LabVIEW		
getCrosstalkThreshold.vi		

getCurrentOutputVoltage

This function gets the current Voltage which is applied to the Piezo.

Please note, that the actual values may be slightly different due to analog/digital converters, amplifiers, etc.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	amplitude	in mV
JSON Method		
method: com.attocube.amc.control.getCurrentOutputVoltage		
params: [axis]		
Result: [errNo, amplitude]		
C-DLL call		
int AMC_control_getCurrentOutputVoltage (int deviceHandle, int axis, int* amplitude)		
Python		
amplitude = [dev].control.getCurrentOutputVoltage (axis)		
Matlab		
[amplitude] = control_getCurrentOutputVoltage (axis)		
C#		
int value = [Device].Control_GetCurrentOutputVoltage (int axis)		
LabVIEW		
getCurrentOutputVoltage.vi		



getEotParameters

Gets the two parameters, that define the behavior of the eot detection (how sensitive respectively how robust it works)

Function specific parameters		
In	axis	[0 1 2] (will be ignored, if minAvgStepSize equals nil)
Out	err	err
	minAvgStepSize_nm	this correpsonds to the "eot_threshold"-parameter
	numOfAvgedSteps	this defines the number of steps, over which the average step size is calculated
JSON Method		
method: com.attocube.amc.control.getEotParameters		
params: [axis]		
Result: [err, minAvgStepSize_nm, numOfAvgedSteps]		
C-DLL call		
int AMC_control_getEotParameters (int deviceHandle, int axis, int* err, int* minAvgStepSize_nm, int* numOfAvgedSteps)		
Python		
err, minAvgStepSize_nm, numOfAvgedSteps = [dev].control.getEotParameters (axis)		
Matlab		
[err, minAvgStepSize_nm, numOfAvgedSteps] = control_getEotParameters (axis)		
C#		
Tuple<int,int,int> value = [Device]. Control_GetEotParameters (int axis)		
LabVIEW		
getEotParameters.vi		

getExternalSensor

This function gets whether the sensor source of closed loop is IDS

It is only available when the feature AMC/IDS closed loop has been activated

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enabled	enabled
JSON Method		
method: com.attocube.amc.control.getExternalSensor		
params: [axis]		
Result: [errNo, enabled]		
C-DLL call		
int AMC_control_getExternalSensor (int deviceHandle, int axis, bool* enabled)		
Python		
enabled = [dev].control.getExternalSensor (axis)		
Matlab		
[enabled] = control_getExternalSensor (axis)		
C#		
bool value = [Device]. Control_GetExternalSensor (int axis)		
LabVIEW		
getExternalSensor.vi		



getFinePositioningRange

This function gets the fine positioning DC-range

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	range	in nm
JSON Method		
method: com.attocube.amc.control.getFinePositioningRange		
params: [axis]		
Result: [errNo, range]		
C-DLL call		
int AMC_control_getFinePositioningRange (int deviceHandle, int axis, int* range)		
Python		
range = [dev].control.getFinePositioningRange(axis)		
Matlab		
[range] = control_getFinePositioningRange (axis)		
C#		
int value = [Device]. Control_GetFinePositioningRange (int axis)		
LabVIEW		
getFinePositioningRange.vi		

getFinePositioningSlewRate

This function gets the fine positioning slew rate

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	slewrates	[0 1 2 3]
JSON Method		
method: com.attocube.amc.control.getFinePositioningSlewRate		
params: [axis]		
Result: [errNo, slewrates]		
C-DLL call		
int AMC_control_getFinePositioningSlewRate (int deviceHandle, int axis, int* slewrates)		
Python		
slewrates = [dev].control.getFinePositioningSlewRate(axis)		
Matlab		
[slewrates] = control_getFinePositioningSlewRate (axis)		
C#		
int value = [Device]. Control_GetFinePositioningSlewRate (int axis)		
LabVIEW		
getFinePositioningSlewRate.vi		



getGndIfIdle

gets whether ground-if-idle shall be active or deactivate
only in AMC300

Function specific parameters		
In	axis	[0 1 2]
Out	err	err
	gndifidleAct	[0: inactive; 1: active]
JSON Method		
method: com.attocube.amc.control.getGndIfIdle		
params: [axis]		
Result: [err, gndifidleAct]		
C-DLL call		
int AMC_control_getGndIfIdle (int deviceHandle, int axis, int* err, int* gndifidleAct)		
Python		
err, gndifidleAct = [dev].control.getGndIfIdle (axis)		
Matlab		
[err, gndifidleAct] = control_getGndIfIdle (axis)		
C#		
Tuple<int,int> value = [Device]. Control_GetGndIfIdle (int axis)		
LabVIEW		
getGndIfIdle.vi		

getMotionControlThreshold

This function gets the threshold range within the closed-loop controlled movement stops to regulate.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	threshold	in pm (nDeg for rotators and gonios)
JSON Method		
method: com.attocube.amc.control.getMotionControlThreshold		
params: [axis]		
Result: [errNo, threshold]		
C-DLL call		
int AMC_control_getMotionControlThreshold (int deviceHandle, int axis, int* threshold)		
Python		
threshold = [dev].control.getMotionControlThreshold(axis)		
Matlab		
[threshold] = control_getMotionControlThreshold (axis)		
C#		
int value = [Device]. Control_GetMotionControlThreshold (int axis)		
LabVIEW		
getMotionControlThreshold.vi		



getMoveDirInverted

This function gets whether the moving direction is inverted on a specific axis. This setting is only relevant for the external sensor. If internal sensor is selected, this parameter is ignored.

Inversion means, that the AMC triggers bwd movement, if fwd-movement is commanded. This is supposed to straighten out upside down moving and measuring coordinate systems

It is only available when the feature AMC/IDS closed loop has been activated.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	mvDirInverted	boolean
JSON Method		
method: com.attocube.amc.control.getMoveDirInverted		
params: [axis]		
Result: [errNo, mvDirInverted]		
C-DLL call		
int AMC_control_getMoveDirInverted (int deviceHandle, int axis, bool* mvDirInverted)		
Python		
mvDirInverted = [dev].control.getMoveDirInverted(axis)		
Matlab		
[mvDirInverted] = control_getMoveDirInverted (axis)		
C#		
bool value = [Device]. Control_GetMoveDirInverted (int axis)		
LabVIEW		
getMoveDirInverted.vi		

getMoveParametersAllAxis

This function reads the parameters for controlling the positioners on all three axes

The parameters are amplitude (V), sawtooth-frequency (Hz), currently on the piezo applied voltage (V)

Function specific parameters		
Out	err	err
	amplitude_ax1_mV	amplitude_ax1_mV
	frequency_ax1_mHz	frequency_ax1_mHz
	appliedVoltage_ax1_mV	currently applied "DC"-Voltage
	amplitude_ax2_mV	amplitude_ax2_mV
	frequency_ax2_mHz	frequency_ax2_mHz
	appliedVoltage_ax2_mV	currently applied "DC"-Voltage
	amplitude_ax3_mV	amplitude_ax3_mV
	frequency_ax3_mHz	frequency_ax3_mHz
	appliedVoltage_ax3_mV	currently applied "DC"-Voltage
JSON Method		
method: com.attocube.amc.control.getMoveParametersAllAxis		
params: []		
Result: [err, amplitude_ax1_mV, frequency_ax1_mHz, appliedVoltage_ax1_mV, amplitude_ax2_mV, frequency_ax2_mHz, appliedVoltage_ax2_mV, amplitude_ax3_mV, frequency_ax3_mHz, appliedVoltage_ax3_mV]		
C-DLL call		
int AMC_control_getMoveParametersAllAxis (int deviceHandle, int* err, int* amplitude_ax1_mV, int* frequency_ax1_mHz, int* appliedVoltage_ax1_mV, int* amplitude_ax2_mV, int* frequency_ax2_mHz, int* appliedVoltage_ax2_mV, int* amplitude_ax3_mV, int* frequency_ax3_mHz, int* appliedVoltage_ax3_mV)		
Python		
err, amplitude_ax1_mV, frequency_ax1_mHz, appliedVoltage_ax1_mV, amplitude_ax2_mV, frequency_ax2_mHz, appliedVoltage_ax2_mV, amplitude_ax3_mV, frequency_ax3_mHz, appliedVoltage_ax3_mV = [dev].control.getMoveParametersAllAxis()		
Matlab		
[err, amplitude_ax1_mV, frequency_ax1_mHz, appliedVoltage_ax1_mV, amplitude_ax2_mV, frequency_ax2_mHz, appliedVoltage_ax2_mV, amplitude_ax3_mV, frequency_ax3_mHz, appliedVoltage_ax3_mV] = control_getMoveParametersAllAxis()		
C#		
Tuple<int,int,int,int,int,int, Tuple<int,int,int,int>> value = [Device]. Control_GetMoveParametersAllAxis()		
LabVIEW		
getMoveParametersAllAxis.vi		



getOutputHealthStatus

This function gets the short circuit status of the selected axis

Possible error codes for the axisHealthStatus are

"73": "HW_SHORT_DETECTED - Unspecific short circuit detected on the drive pins. Axis disabled"

"83": "HW_SCP_DETECTED - Short circuit detected. Axis is disabled and locked."

"84": "HW_SCP_UNCONNECTED - Short circuit detected while no positioner connected. Axis is disabled and locked."

"85": "HW_DC_CURRENT_TOO_HIGH - DC current too high during axis activation. Axis is disabled and locked."

"86": "HW_DC_OVERCURRENT_AXIS - DC current too high during axis operation. Axis is disabled and locked."

"87": "HW_DC_OVERCURRENT_ERROR - Global over current detected during operation. All axes are disabled and locked."

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	axisHealthStatus	(0 = OK, not 0 = error code for detected short circuit condition)
JSON Method		
method: com.attocube.amc.control.getOutputHealthStatus		
params: [axis]		
Result: [errNo, axisHealthStatus]		
C-DLL call		
int AMC_control_getOutputHealthStatus (int deviceHandle, int axis, int* axisHealthStatus)		
Python		
axisHealthStatus = [dev].control.getOutputHealthStatus(axis)		
Matlab		
[axisHealthStatus] = control_getOutputHealthStatus(axis)		
C#		
int value = [Device].Control_GetOutputHealthStatus(int axis)		
LabVIEW		
getOutputHealthStatus.vi		

getPositionsAndVoltages

Simultaneously get 3 axes positions as well as the DC offset

position-value is -2147483648 nm, if sensor is disabled or no positioner is connected.

Function specific parameters		
Out	errNo	errNo
	pos1	position of axis 1
	pos2	position of axis 2
	pos3	position of axis 3
	val1	dc voltage of of axis 1 in mV
	val2	dc voltage of of axis 2 in mV
	val3	dc voltage of of axis 3 in mV
JSON Method		
method: com.attocube.amc.control.getPositionsAndVoltages		
params: []		
Result: [errNo, pos1, pos2, pos3, val1, val2, val3]		
C-DLL call		
int AMC_control_getPositionsAndVoltages (int deviceHandle, double* pos1, double* pos2, double* pos3, int* val1, int* val2, int* val3)		
Python		
pos1, pos2, pos3, val1, val2, val3 = [dev].control.getPositionsAndVoltages()		
Matlab		
[pos1, pos2, pos3, val1, val2, val3] = control_getPositionsAndVoltages()		
C#		
Tuple<double,double,double,int,int,int> value = [Device].Control_GetPositionsAndVoltages()		
LabVIEW		
getPositionAndVoltages.vi		



getReferencePosition

This function gets the reference position of the selected axis.

It can only be used in conjunction with NUM.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	position	position: For linear type actors the position is defined in nm for goniometer an rotator type actors it is μ° .
JSON Method		
method: com.attocube.amc.control.getReferencePosition		
params: [axis]		
Result: [errNo, position]		
C-DLL call		
int AMC_control_getReferencePosition (int deviceHandle, int axis, int* position)		
Python		
position = [dev].control.getReferencePosition(axis)		
Matlab		
[position] = control_getReferencePosition (axis)		
C#		
int value = [Device]. Control_GetReferencePosition (int axis)		
LabVIEW		
getReferencePosition.vi		

getSensorDirection

This function gets whether the IDS sensor source of closed loop is inverted

It is only available when the feature AMC/IDS closed loop has been activated

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	inverted	boolean
JSON Method		
method: com.attocube.amc.control.getSensorDirection		
params: [axis]		
Result: [errNo, inverted]		
C-DLL call		
int AMC_control_getSensorDirection (int deviceHandle, int axis, bool* inverted)		
Python		
inverted = [dev].control.getSensorDirection (axis)		
Matlab		
[inverted] = control_getSensorDirection (axis)		
C#		
bool value = [Device]. Control_GetSensorDirection (int axis)		
LabVIEW		
getSensorDirection.vi		



getSensorEnabled

Get sensor power supply status

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	value	true if enabled, false otherwise
JSON Method		
method: com.attocube.amc.control.getSensorEnabled		
params: [axis]		
Result: [errNo, value]		
C-DLL call		
int AMC_control_getSensorEnabled (int deviceHandle, int axis, bool* value)		
Python		
value = [dev].control.getSensorEnabled(axis)		
Matlab		
[value] = control_getSensorEnabled (axis)		
C#		
bool value = [Device]. Control_GetSensorEnabled (int axis)		
LabVIEW		
getSensorEnabled.vi		

getStatusMovingAllAxes

Get Status of all axes, see getStatusMoving for coding of the values

Function specific parameters		
Out	errNo	errNo
	moving1	status of axis 1
	moving2	status of axis 2
	moving3	status of axis 3
JSON Method		
method: com.attocube.amc.control.getStatusMovingAllAxes		
params: []		
Result: [errNo, moving1, moving2, moving3]		
C-DLL call		
int AMC_control_getStatusMovingAllAxes (int deviceHandle, int* moving1, int* moving2, int* moving3)		
Python		
moving1, moving2, moving3 = [dev].control.getStatusMovingAllAxes()		
Matlab		
[moving1, moving2, moving3] = control_getStatusMovingAllAxes ()		
C#		
Tuple<int,int,int> value = [Device]. Control_GetStatusMovingAllAxes ()		
LabVIEW		
getStatusMovingAllAxes.vi		



resetOutputHealthStatus

This function resets the short circuit status of the selected axis.

Function specific parameters		
In	axis	[0 1 2]
	solution	Short description of taken measures to resolve the short circuit (min. 20 characters and 3 words, maximum 499 characters)
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.resetOutputHealthStatus		
params: [axis, solution]		
Result: [errNo]		
C-DLL call		
int AMC_control_resetOutputHealthStatus (int deviceHandle, int axis, const char* solution)		
Python		
[dev].control.resetOutputHealthStatus (axis, solution)		
Matlab		
[] = control_resetOutputHealthStatus (axis, solution)		
C#		
void value = [Device]. Control_ResetOutputHealthStatus (int axis, string solution)		
LabVIEW		
resetOutputHealthStatus.vi		

searchReferencePosition

This function searches for the reference position of the selected axis.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.searchReferencePosition		
params: [axis]		
Result: [errNo]		
C-DLL call		
int AMC_control_searchReferencePosition (int deviceHandle, int axis)		
Python		
[dev].control.searchReferencePosition (axis)		
Matlab		
[] = control_searchReferencePosition (axis)		
C#		
void value = [Device]. Control_SearchReferencePosition (int axis)		
LabVIEW		
searchReferencePosition.vi		



setActorParametersByName

This function sets the name for the positioner on the selected axis. The possible names can be retrieved by executing `getPositionersList`

Function specific parameters		
In	axis	[0 1 2]
	actorname	name of the actor
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setActorParametersByName		
params: [axis, actorname]		
Result: [errNo]		
C-DLL call		
int AMC_control_setActorParametersByName (int deviceHandle, int axis, const char* actorname)		
Python		
[dev].control.setActorParametersByName (axis, actorname)		
Matlab		
[] = control_setActorParametersByName (axis, actorname)		
C#		
void value = [Device]. Control_SetActorParametersByName (int axis, string actorname)		
LabVIEW		
setActorParametersByName.vi		

setActorParametersJson

This function can be used to change several actor parameters with one call. All key-value-pairs in the second argument will be set to the given value.

The other parameters remain unchanged. This function can not be used to load a different positioner-type and all its related parameters.

Please use setActorParametersByName to do that.

Function specific parameters		
In	axis	[0 1 2]
	json_dict	dict with override params
Out	errNo	errorCode
JSON Method		
method: com.attocube.amc.control.setActorParametersJson		
params: [axis, json_dict]		
Result: [errNo]		
C-DLL call		
int AMC_control_setActorParametersJson (int deviceHandle, int axis, const char* json_dict)		
Python		
[dev].control.setActorParametersJson (axis, json_dict)		
Matlab		
[] = control_setActorParametersJson (axis, json_dict)		
C#		
void value = [Device]. Control_SetActorParametersJson (int axis, string json_dict)		
LabVIEW		
setActorParametersJson.vi		



setActorSensitivity

Control the actor parameter closed loop sensitivity

Function specific parameters		
In	axis	[0 1 2]
	sensitivity	[0 (fast approach) .. 10 (slow approach)] The approach behaviour changes exponentially with the sensitivity
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setActorSensitivity		
params: [axis, sensitivity]		
Result: [errNo]		
C-DLL call		
int AMC_control_setActorSensitivity (int deviceHandle, int axis, int sensitivity)		
Python		
[dev].control.setActorSensitivity (axis, sensitivity)		
Matlab		
[] = control_setActorSensitivity (axis, sensitivity)		
C#		
void value = [Device]. Control_SetActorSensitivity (int axis, int sensitivity)		
LabVIEW		
setActorSensitivity.vi		

setAutoMeasure

This function enables/disables the automatic C/R measurement on axis enable

Function specific parameters		
In	axis	[0 1 2]
	enable	true: enable automeasurement, false: disable automeasurement
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setAutoMeasure		
params: [axis, enable]		
Result: [errNo]		
C-DLL call		
int AMC_control_setAutoMeasure (int deviceHandle, int axis, bool enable)		
Python		
[dev].control.setAutoMeasure (axis, enable)		
Matlab		
[] = control_setAutoMeasure (axis, enable)		
C#		
void value = [Device]. Control_SetAutoMeasure (int axis, bool enable)		
LabVIEW		
setAutoMeasure.vi		



setAxesSensorSources

This function sets the sensor source according to the config passed to the function. An axis can be set to the IDS as sensor source ("external") or to the sensor of the positioner in use ("internal"; NUM or RES).

Please note, that not all possible axis configurations are supported. Refer to the parameter description of extSensCfg for the supported configurations.

It is only available when the feature AMC/IDS closed loop has been activated

Function specific parameters		
In	extSensCfg	[0: intern,intern,intern 1: extern,extern,extern 2: extern,extern,intern 3: intern,intern,extern]
Out	errNo	Warning code, can be converted into a string using the errorNumberToString function
JSON Method		
method: com.attocube.amc.control.setAxesSensorSources		
params: [extSensCfg]		
Result: [errNo]		
C-DLL call		
int AMC_control_setAxesSensorSources (int deviceHandle, int extSensCfg)		
Python		
[dev].control.setAxesSensorSources(extSensCfg)		
Matlab		
[] = control_setAxesSensorSources(extSensCfg)		
C#		
void value = [Device].Control_SetAxesSensorSources(int extSensCfg)		
LabVIEW		
setAxesSensorSources.vi		

setControlAmplitude

This function sets the amplitude of the actuator signal of the selected axis.

Function specific parameters		
In	axis	[0 1 2]
	amplitude	in mV
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setControlAmplitude		
params: [axis, amplitude]		
Result: [errNo]		
C-DLL call		
int AMC_control_setControlAmplitude (int deviceHandle, int axis, int amplitude)		
Python		
[dev].control.setControlAmplitude (axis, amplitude)		
Matlab		
[] = control_setControlAmplitude (axis, amplitude)		
C#		
void value = [Device]. Control_SetControlAmplitude (int axis, int amplitude)		
LabVIEW		
setControlAmplitude.vi		



setControlAutoReset

This function controls the behaviour, that the current position (which can be retrieved with a getPosition-call) is reset every time the reference mark of the positioner is detected.

This function can only be used in conjunction with NUM.

Function specific parameters		
In	axis	[0 1 2]
	enable	boolean
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setControlAutoReset		
params: [axis, enable]		
Result: [errNo]		
C-DLL call		
int AMC_control_setControlAutoReset (int deviceHandle, int axis, bool enable)		
Python		
[dev].control.setControlAutoReset (axis, enable)		
Matlab		
[] = control_setControlAutoReset (axis, enable)		
C#		
void value = [Device]. Control_SetControlAutoReset (int axis, bool enable)		
LabVIEW		
setControlAutoReset.vi		

setControlFixOutputVoltage

This function sets the DC level output of the selected axis.

Function specific parameters		
In	axis	[0 1 2]
	amplitude_mv	in mV
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setControlFixOutputVoltage		
params: [axis, amplitude_mv]		
Result: [errNo]		
C-DLL call		
int AMC_control_setControlFixOutputVoltage (int deviceHandle, int axis, int amplitude_mv)		
Python		
[dev].control.setControlFixOutputVoltage (axis, amplitude_mv)		
Matlab		
[] = control_setControlFixOutputVoltage (axis, amplitude_mv)		
C#		
void value = [Device]. Control_SetControlFixOutputVoltage (int axis, int amplitude_mv)		
LabVIEW		
setControlFixOutputVoltage.vi		



setControlFrequency

This function sets the frequency of the actuator signal of the selected axis.

Note: Approximate the slewrate of the motion controller according to Input Frequency

Function specific parameters		
In	axis	[0 1 2]
	frequency	in mHz
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setControlFrequency		
params: [axis, frequency]		
Result: [errNo]		
C-DLL call		
int AMC_control_setControlFrequency (int deviceHandle, int axis, int frequency)		
Python		
[dev].control.setControlFrequency (axis, frequency)		
Matlab		
[] = control_setControlFrequency (axis, frequency)		
C#		
void value = [Device]. Control_SetControlFrequency (int axis, int frequency)		
LabVIEW		
setControlFrequency.vi		

setControlMove

This function sets the approach of the selected axis' positioner to the target position.

Function specific parameters		
In	axis	[0 1 2]
	enable	boolean true: enable the approach , false: disable the approach
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setControlMove		
params: [axis, enable]		
Result: [errNo]		
C-DLL call		
int AMC_control_setControlMove (int deviceHandle, int axis, bool enable)		
Python		
[dev].control.setControlMove (axis, enable)		
Matlab		
[] = control_setControlMove (axis, enable)		
C#		
void value = [Device]. Control_SetControlMove (int axis, bool enable)		
LabVIEW		
setControlMove.vi		



setControlOutput

This function sets the activation status of the axis.

Please make sure, a positioner is connected correctly to the corresponding axis before enabling an axis.

Function specific parameters		
In	axis	[0 1 2]
	enable	true: enable drives, false: disable drives
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setControlOutput		
params: [axis, enable]		
Result: [errNo]		
C-DLL call		
int AMC_control_setControlOutput (int deviceHandle, int axis, bool enable)		
Python		
[dev].control.setControlOutput (axis, enable)		
Matlab		
[] = control_setControlOutput (axis, enable)		
C#		
void value = [Device]. Control_SetControlOutput (int axis, bool enable)		
LabVIEW		
setControlOutput.vi		

setControlReferenceAutoUpdate

This function sets the status of whether the reference position is updated when the reference mark is hit.

When this function is disabled, the reference marking will be considered only the first time and after then ignored.

This function can only be used in conjunction with NUM.

Function specific parameters		
In	axis	[0 1 2]
	enable	boolean
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setControlReferenceAutoUpdate		
params: [axis, enable]		
Result: [errNo]		
C-DLL call		
int AMC_control_setControlReferenceAutoUpdate (int deviceHandle, int axis, bool enable)		
Python		
[dev].control.setControlReferenceAutoUpdate (axis, enable)		
Matlab		
[] = control_setControlReferenceAutoUpdate (axis, enable)		
C#		
void value = [Device]. Control_SetControlReferenceAutoUpdate (int axis, bool enable)		
LabVIEW		
setControlReferenceAutoUpdate.vi		



setControlTargetRange

This function sets the range around the target position in which the flag "In Target Range" (see VIII.7.a) becomes active.

Please note, that this setting is only used to identify the target position, it has no influence on the closed loop control.

Function specific parameters		
In	axis	[0 1 2]
	range	in nm
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setControlTargetRange		
params: [axis, range]		
Result: [errNo]		
C-DLL call		
int AMC_control_setControlTargetRange (int deviceHandle, int axis, int range)		
Python		
[dev].control.setControlTargetRange (axis, range)		
Matlab		
[] = control_setControlTargetRange (axis, range)		
C#		
void value = [Device]. Control_SetControlTargetRange (int axis, int range)		
LabVIEW		
setControlTargetRange.vi		

setCrosstalkThreshold

This function sets the threshold range to avoid axis-crosstalk and slip phase time which is used while moving another axis

Function specific parameters		
In	axis	[0 1 2]
	threshold	[max:2000000000][pm]; has to be greater than the motion-control-threshold
	slipphasetime	[min=0,max=65535][us] time after slip phase which is waited until the controller acts again
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setCrosstalkThreshold		
params: [axis, threshold, slipphasetime]		
Result: [errNo]		
C-DLL call		
int AMC_control_setCrosstalkThreshold (int deviceHandle, int axis, int threshold, int slipphasetime)		
Python		
[dev].control.setCrosstalkThreshold (axis, threshold, slipphasetime)		
Matlab		
[] = control_setCrosstalkThreshold (axis, threshold, slipphasetime)		
C#		
void value = [Device]. Control_SetCrosstalkThreshold (int axis, int threshold, int slipphasetime)		
LabVIEW		
setCrosstalkThreshold.vi		



setEotParameters

Sets the two parameters, that define the behavior of the eot detection (how sensitive respectively how robust it works)

Function specific parameters		
In	axis	[0 1 2] (will be ignored, if minAvgStepSize equals nil)
	minAvgStepSize_nm	this corresponds to the "eot_threshold"-parameter
	numOfAvgedSteps	this defines the number of steps, over which the average step size is calculated
Out	err	err
JSON Method		
method: com.attocube.amc.control.setEotParameters		
params: [axis, minAvgStepSize_nm, numOfAvgedSteps]		
Result: [err]		
C-DLL call		
int AMC_control_setEotParameters (int deviceHandle, int axis, int minAvgStepSize_nm, int numOfAvgedSteps, int* err)		
Python		
err = [dev].control.setEotParameters(axis, minAvgStepSize_nm, numOfAvgedSteps)		
Matlab		
[err] = control_setEotParameters (axis, minAvgStepSize_nm, numOfAvgedSteps)		
C#		
int value = [Device]. Control_SetEotParameters (int axis, int minAvgStepSize_nm, int numOfAvgedSteps)		
LabVIEW		
setEotParameters.vi		

setFinePositioningRange

This function sets the fine positioning DC-range

Function specific parameters		
In	axis	[0 1 2]
	range	in nm
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setFinePositioningRange		
params: [axis, range]		
Result: [errNo]		
C-DLL call		
int AMC_control_setFinePositioningRange (int deviceHandle, int axis, int range)		
Python		
[dev].control.setFinePositioningRange (axis, range)		
Matlab		
[] = control_setFinePositioningRange (axis, range)		
C#		
void value = [Device]. Control_SetFinePositioningRange (int axis, int range)		
LabVIEW		
setFinePositioningRange.vi		



setFinePositioningSlewRate

This function sets the fine positioning slew rate

Function specific parameters		
In	axis	[0 1 2]
	slewrates	[0 1 2 3]
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setFinePositioningSlewRate		
params: [axis, slewrates]		
Result: [errNo]		
C-DLL call		
int AMC_control_setFinePositioningSlewRate (int deviceHandle, int axis, int slewrates)		
Python		
[dev].control.setFinePositioningSlewRate (axis, slewrates)		
Matlab		
[] = control_setFinePositioningSlewRate (axis, slewrates)		
C#		
void value = [Device]. Control_SetFinePositioningSlewRate (int axis, int slewrates)		
LabVIEW		
setFinePositioningSlewRate.vi		

setGndIfIdle

sets the ground-if-idle feature active or inactive

Please note, that changing this setting to false while the axis is active will deactivate the axis

This function cannot be executed during open- or closed-loop movement on the given axis

This feature is only available for AMC300.

Function specific parameters		
In	axis	[0 1 2]
	gndifidleAct	[0: inactive; 1: active]
Out	err	err
JSON Method		
method: com.attocube.amc.control.setGndIfIdle		
params: [axis, gndifidleAct]		
Result: [err]		
C-DLL call		
int AMC_control_setGndIfIdle (int deviceHandle, int axis, int gndifidleAct, int* err)		
Python		
err = [dev].control.setGndIfIdle(axis, gndifidleAct)		
Matlab		
[err] = control_setGndIfIdle (axis, gndifidleAct)		
C#		
int value = [Device]. Control_SetGndIfIdle (int axis, int gndifidleAct)		
LabVIEW		
setGndIfIdle.vi		



setMotionControlThreshold

This function sets the threshold range within the closed-loop controlled movement stops to regulate. Default depends on connected sensor type

Function specific parameters		
In	axis	[0 1 2]
	threshold	in pm (nDeg for rotators and gonios)
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setMotionControlThreshold		
params: [axis, threshold]		
Result: [errNo]		
C-DLL call		
int AMC_control_setMotionControlThreshold (int deviceHandle, int axis, int threshold)		
Python		
[dev].control.setMotionControlThreshold (axis, threshold)		
Matlab		
[] = control_setMotionControlThreshold (axis, threshold)		
C#		
void value = [Device]. Control_SetMotionControlThreshold (int axis, int threshold)		
LabVIEW		
setMotionControlThreshold.vi		

setMoveDirInverted

This function sets whether the moving direction shall be inverted on a specific axis. This parameter can only be set, if the external sensor is selected on this axis. Please refer to setAxesSensorSources for the external sensor setting.

Inversion means, that the AMC e.g. triggers bwd movement, if fwd-movement is commanded. This is supposed to straighten out upside down moving and measuring coordinate systems

It is only available when the feature AMC/IDS closed loop has been activated.

Function specific parameters		
In	axis	[0 1 2]
	mvDirInverted	
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setMoveDirInverted		
params: [axis, mvDirInverted]		
Result: [errNo]		
C-DLL call		
int AMC_control_setMoveDirInverted (int deviceHandle, int axis, bool mvDirInverted)		
Python		
[dev].control.setMoveDirInverted (axis, mvDirInverted)		
Matlab		
[] = control_setMoveDirInverted (axis, mvDirInverted)		
C#		
void value = [Device]. Control_SetMoveDirInverted (int axis, bool mvDirInverted)		
LabVIEW		
setMoveDirInverted.vi		



setReset

This function resets the actual position of the selected axis given by the NUM sensor to zero and marks the reference position as invalid.

It does not work for RES positioners and positions read by IDS.

For IDS, use `com.attocube.ids.displacement.resetAxis()` or `com.attocube.amc.amcids.resetIdsAxis()` instead.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
JSON Method		
method: <code>com.attocube.amc.control.setReset</code>		
params: [axis]		
Result: [errNo]		
C-DLL call		
<code>int AMC_control_setReset(int deviceHandle, int axis)</code>		
Python		
<code>[dev].control.setReset(axis)</code>		
Matlab		
<code>[] = control_setReset(axis)</code>		
C#		
<code>void value = [Device].Control_SetReset(int axis)</code>		
LabVIEW		
setReset.vi		

setSensorEnabled

Set sensor power supply status, can be switched off to save heat generated by sensor [NUM or RES]; no effect on IDS in AMC-IDS-CL feature

Positions retrieved will be invalid when activating this, so closed-loop control should be switched off beforehand

Function specific parameters		
In	axis	[0 1 2]
	value	true if enabled, false otherwise
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.control.setSensorEnabled		
params: [axis, value]		
Result: [errNo]		
C-DLL call		
int AMC_control_setSensorEnabled (int deviceHandle, int axis, bool value)		
Python		
[dev].control.setSensorEnabled (axis, value)		
Matlab		
[] = control_setSensorEnabled (axis, value)		
C#		
void value = [Device]. Control_SetSensorEnabled (int axis, bool value)		
LabVIEW		
setSensorEnabled.vi		



3.5. Description

checkChassisNbr

Get Chassis and Slot Number, only works when AMC is within a Rack

Function specific parameters		
Out	errNo	errorCode
	slotNbr	slotNbr
	chassisNbr	chassisNbr
JSON Method		
method: com.attocube.amc.description.checkChassisNbr		
params: []		
Result: [errNo, slotNbr, chassisNbr]		
C-DLL call		
int AMC_description_checkChassisNbr (int deviceHandle, int* slotNbr, int* chassisNbr)		
Python		
slotNbr, chassisNbr = [dev].description.checkChassisNbr()		
Matlab		
[slotNbr, chassisNbr] = description_checkChassisNbr ()		
C#		
Tuple<int,int> value = [Device]. Description_CheckChassisNbr ()		
LabVIEW		
checkChassisNbr.vi		

getDeviceType

This function gets the device type based on its EEPROM configuration.

Function specific parameters		
Out	errNo	errNo
	devicetype	Device name (AMC100, AMC150, AMC300) with attached feature (AMC100\\NUM, AMC100\\NUM\\PRO)
JSON Method		
method: com.attocube.amc.description.getDeviceType		
params: []		
Result: [errNo, devicetype]		
C-DLL call		
int AMC_description_getDeviceType (int deviceHandle, char* devicetype, int size0)		
Python		
devicetype = [dev].description.getDeviceType()		
Matlab		
[devicetype] = description_getDeviceType ()		
C#		
string value = [Device]. Description_GetDeviceType ()		
LabVIEW		
getDeviceType.vi		



getFeaturesActivated

Get the activated features and return as a string

Function specific parameters		
Out	errNo	errNo
	features	activated on device concatenated by comma e.g. Closed loop Operation, Pro, Wireless Controller, IO
JSON Method		
method: com.attocube.amc.description.getFeaturesActivated		
params: []		
Result: [errNo, features]		
C-DLL call		
int AMC_description_getFeaturesActivated (int deviceHandle, char* features, int size0)		
Python		
features = [dev].description.getFeaturesActivated()		
Matlab		
[features] = description_getFeaturesActivated ()		
C#		
string value = [Device]. Description_GetFeaturesActivated ()		
LabVIEW		
getFeaturesActivated.vi		

getPositionersList

This function reads the actor names that can be connected to the device.

Function specific parameters		
Out	errNo	errNo
	PositionersList	PositionersList
JSON Method		
method: com.attocube.amc.description.getPositionersList		
params: []		
Result: [errNo, PositionersList]		
C-DLL call		
int AMC_description_getPositionersList (int deviceHandle, char* PositionersList, int size0)		
Python		
PositionersList = [dev].description.getPositionersList()		
Matlab		
[PositionersList] = description_getPositionersList ()		
C#		
string value = [Device]. Description_GetPositionersList ()		
LabVIEW		
getPositionersList.vi		



3.6. Diagnostic

getDiagnosticPower

Returns the current power consumption

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	power	power
JSON Method		
method: com.attocube.amc.diagnostic.getDiagnosticPower		
params: [axis]		
Result: [errNo, power]		
C-DLL call		
int AMC_diagnostic_getDiagnosticPower(int deviceHandle, int axis, int* power)		
Python		
power = [dev].diagnostic.getDiagnosticPower(axis)		
Matlab		
[power] = diagnostic_getDiagnosticPower(axis)		
C#		
int value = [Device].Diagnostic_GetDiagnosticPower(int axis)		
LabVIEW		
getDiagnosticPower.vi		

getDiagnosticResults

Returns the results of the last diagnostic run and an error, if there was no run, it is currently running or the run failed

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	capacity	in nF
	resistance	in Ohm
JSON Method		
method: com.attocube.amc.diagnostic.getDiagnosticResults		
params: [axis]		
Result: [errNo, capacity, resistance]		
C-DLL call		
int AMC_diagnostic_getDiagnosticResults (int deviceHandle, int axis, int* capacity, int* resistance)		
Python		
capacity, resistance = [dev].diagnostic.getDiagnosticResults (axis)		
Matlab		
[capacity, resistance] = diagnostic_getDiagnosticResults (axis)		
C#		
Tuple<int,int> value = [Device].Diagnostic_GetDiagnosticResults (int axis)		
LabVIEW		
getDiagnosticResults.vi		



getDiagnosticTemperature

Returns the current axis temperature

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	temperature	temperature
JSON Method		
method: com.attocube.amc.diagnostic.getDiagnosticTemperature		
params: [axis]		
Result: [errNo, temperature]		
C-DLL call		
int AMC_diagnostic_getDiagnosticTemperature (int deviceHandle, int axis, int* temperature)		
Python		
temperature = [dev].diagnostic.getDiagnosticTemperature(axis)		
Matlab		
[temperature] = diagnostic_getDiagnosticTemperature (axis)		
C#		
int value = [Device]. Diagnostic_GetDiagnosticTemperature (int axis)		
LabVIEW		
getDiagnosticTemperature.vi		

startDiagnostic

Start the diagnosis procedure for the given axis

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.diagnostic.startDiagnostic		
params: [axis]		
Result: [errNo]		
C-DLL call		
int AMC_diagnostic_startDiagnostic (int deviceHandle, int axis)		
Python		
[dev].diagnostic.startDiagnostic(axis)		
Matlab		
[] = diagnostic_startDiagnostic(axis)		
C#		
void value = [Device]. Diagnostic_StartDiagnostic (int axis)		
LabVIEW		
startDiagnostic.vi		



3.7. Move

getControlContinuousBkwd

This function gets the axis' movement status in backward direction.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enabled	true if movement backward is active , false otherwise
JSON Method		
method: com.attocube.amc.move.getControlContinuousBkwd		
params: [axis]		
Result: [errNo, enabled]		
C-DLL call		
int AMC_move_getControlContinuousBkwd(int deviceHandle, int axis, bool* enabled)		
Python		
enabled = [dev].move.getControlContinuousBkwd(axis)		
Matlab		
[enabled] = move_getControlContinuousBkwd(axis)		
C#		
bool value = [Device].Move_GetControlContinuousBkwd(int axis)		
LabVIEW		
getControlContinuousBkwd.vi		

getControlContinuousFwd

This function gets the axis' movement status in positive direction.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enabled	true if movement Fwd is active, false otherwise
JSON Method		
method: com.attocube.amc.move.getControlContinuousFwd		
params: [axis]		
Result: [errNo, enabled]		
C-DLL call		
int AMC_move_getControlContinuousFwd (int deviceHandle, int axis, bool* enabled)		
Python		
enabled = [dev].move.getControlContinuousFwd(axis)		
Matlab		
[enabled] = move_getControlContinuousFwd (axis)		
C#		
bool value = [Device]. Move_GetControlContinuousFwd (int axis)		
LabVIEW		
getControlContinuousFwd.vi		



getControlEotOutputDeactive

This function gets the output applied to the selected axis on the end of travel. /PRO feature.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enabled	If true, the output of the axis will be deactivated on positive EOT detection.
JSON Method		
method: com.attocube.amc.move.getControlEotOutputDeactive		
params: [axis]		
Result: [errNo, enabled]		
C-DLL call		
int AMC_move_getControlEotOutputDeactive (int deviceHandle, int axis, bool* enabled)		
Python		
enabled = [dev].move.getControlEotOutputDeactive(axis)		
Matlab		
[enabled] = move_getControlEotOutputDeactive (axis)		
C#		
bool value = [Device]. Move_GetControlEotOutputDeactive (int axis)		
LabVIEW		
getControlEotOutputDeactive.vi		

getControlTargetPosition

This function gets the target position for the movement on the selected axis.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	position	defined in nm for goniometer an rotator type actors it is μ° .
JSON Method		
method: com.attocube.amc.move.getControlTargetPosition		
params: [axis]		
Result: [errNo, position]		
C-DLL call		
int AMC_move_getControlTargetPosition (int deviceHandle, int axis, double* position)		
Python		
position = [dev].move.getControlTargetPosition(axis)		
Matlab		
[position] = move_getControlTargetPosition(axis)		
C#		
double value = [Device].Move_GetControlTargetPosition(int axis)		
LabVIEW		
getControlTargetPosition.vi		



getGroundAxis

Checks if the axis piezo drive is actively grounded.

Please note, that this function only returns whether or not the axis is grounded. To get the axis activation status, please use getControlOutput.

This function is only available on AMC300 devices.

Function specific parameters		
In	axis	motion controller axis [0 1 2]
Out	errNo	0 or error
	grounded	true or false
JSON Method		
method: com.attocube.amc.move.getGroundAxis		
params: [axis]		
Result: [errNo, grounded]		
C-DLL call		
int AMC_move_getGroundAxis (int deviceHandle, int axis, bool* grounded)		
Python		
grounded = [dev].move.getGroundAxis (axis)		
Matlab		
[grounded] = move_getGroundAxis (axis)		
C#		
bool value = [Device]. Move_GetGroundAxis (int axis)		
LabVIEW		
getGroundAxis.vi		

getGroundAxisAutoOnTarget

Pull axis piezo drive to GND if positioner is in ground target range
only in AMC300

Function specific parameters		
In	axis	motion controller axis [0 1 2]
Out	errNo	0 or error
	value	true or false
JSON Method		
method: com.attocube.amc.move.getGroundAxisAutoOnTarget		
params: [axis]		
Result: [errNo, value]		
C-DLL call		
int AMC_move_getGroundAxisAutoOnTarget (int deviceHandle, int axis, bool* value)		
Python		
value = [dev].move.getGroundAxisAutoOnTarget(axis)		
Matlab		
[value] = move_getGroundAxisAutoOnTarget (axis)		
C#		
bool value = [Device]. Move_GetGroundAxisAutoOnTarget (int axis)		
LabVIEW		
getGroundAxisAutoOnTarget.vi		



getGroundTargetRange

Retrieves the range around the target position in which the auto grounding becomes active.
only in AMC300

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	targetrange	in nm
JSON Method		
method: com.attocube.amc.move.getGroundTargetRange		
params: [axis]		
Result: [errNo, targetrange]		
C-DLL call		
int AMC_move_getGroundTargetRange (int deviceHandle, int axis, int* targetrange)		
Python		
targetrange = [dev].move.getGroundTargetRange(axis)		
Matlab		
[targetrange] = move_getGroundTargetRange (axis)		
C#		
int value = [Device]. Move_GetGroundTargetRange (int axis)		
LabVIEW		
getGroundTargetRange.vi		

getNSteps

This function gets the number of Steps in desired direction.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	nbrstep	nbrstep
JSON Method		
method: com.attocube.amc.move.getNSteps		
params: [axis]		
Result: [errNo, nbrstep]		
C-DLL call		
int AMC_move_getNSteps (int deviceHandle, int axis, int* nbrstep)		
Python		
nbrstep = [dev].move.getNSteps(axis)		
Matlab		
[nbrstep] = move_getNSteps(axis)		
C#		
int value = [Device].Move_GetNSteps(int axis)		
LabVIEW		
getNSteps.vi		



getPosition

This function gets the current position of the positioner on the selected axis

Note, that this function returns the position '0' in an AMC-IDS-Closedloop setup with IDS as sensor-source, if the IDS reports an error (e.g. beam-interrupt)

The axis on the web application are indexed from 1 to 3

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	position	defined in nm for goniometer an rotator type actors it is μ° .
JSON Method		
method: com.attocube.amc.move.getPosition		
params: [axis]		
Result: [errNo, position]		
C-DLL call		
int AMC_move_getPosition (int deviceHandle, int axis, double* position)		
Python		
position = [dev].move.getPosition (axis)		
Matlab		
[position] = move_getPosition (axis)		
C#		
double value = [Device]. Move_GetPosition (int axis)		
LabVIEW		
getPosition.vi		

getPositionAllAxis

This function returns the position of all three axes in nm

If there is no positioner on one axis or the sensor is disabled, it returns -2147483648 nm instead

Function specific parameters		
Out	err	err
	pos_nm_x	position of the x-axis in nm
	pos_nm_y	position of the y-axis in nm
	pos_nm_z	position of the z-axis in nm
JSON Method		
method: com.attocube.amc.move.getPositionAllAxis		
params: []		
Result: [err, pos_nm_x, pos_nm_y, pos_nm_z]		
C-DLL call		
int AMC_move_getPositionAllAxis (int deviceHandle, int* err, double* pos_nm_x, double* pos_nm_y, double* pos_nm_z)		
Python		
err, pos_nm_x, pos_nm_y, pos_nm_z = [dev].move.getPositionAllAxis()		
Matlab		
[err, pos_nm_x, pos_nm_y, pos_nm_z] = move_getPositionAllAxis()		
C#		
Tuple<int,double,double,double> value = [Device]. Move_GetPositionAllAxis()		
LabVIEW		
getPositionAllAxis.vi		



getPositionWithTime

This function gets the current position of the positioner and provides time-information to the position.

The time information refers to the elapsed time since the last reboot of the device in microseconds.

The axis on the web application are indexed from 1 to 3

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	Monotnoic_time_usec	Monotnoic_time_usec: elapsed time in microseconds since last reboot of device
	position	defined in nm for goniometer an rotator type actors it is μ° .
JSON Method		
method: com.attocube.amc.move.getPositionWithTime		
params: [axis]		
Result: [errNo, Monotnoic_time_usec, position]		
C-DLL call		
int AMC_move_getPositionWithTime (int deviceHandle, int axis, double* Monotnoic_time_usec, double* position)		
Python		
Monotnoic_time_usec, position = [dev].move.getPositionWithTime(axis)		
Matlab		
[Monotnoic_time_usec, position] = move_getPositionWithTime (axis)		
C#		
Tuple<double,double> value = [Device]. Move_GetPositionWithTime (int axis)		
LabVIEW		
getPositionWithTime.vi		

getPositionWithTime_32Bit

This function gets the current position of the positioner and provides time-information to the position.

The time information refers to the elapsed time since the last reboot of the device.

This function is to be used if 64 bit numbers are not supported by your platform

It returns two 32 bit numbers as time information instead of one number of 64 Bit (see description of return parameters).

The axis on the web application are indexed from 1 to 3

Function specific parameters		
Ξ	axis	[0 1 2]
Out	errNo	errNo
	Monotonic_time_sec	Monotonic_time_sec: seconds passed since last reboot of device
	Monotonic_time_nsec	Monotonic_time_nsec: fractional seconds of Monotonic_time_sec
	position	defined in nm for goniometer an rotator type actors it is μ° .
JSON Method		
method: com.attocube.amc.move.getPositionWithTime_32Bit		
params: [axis]		
Result: [errNo, Monotonic_time_sec, Monotonic_time_nsec, position]		
C-DLL call		
int AMC_move_getPositionWithTime_32Bit (int deviceHandle, int axis, int* Monotonic_time_sec, int* Monotonic_time_nsec, double* position)		
Python		
Monotonic_time_sec, Monotonic_time_nsec, position = [dev].move.getPositionWithTime_32Bit(axis)		
Matlab		
[Monotonic_time_sec, Monotonic_time_nsec, position] = move_getPositionWithTime_32Bit(axis)		
C#		
Tuple<int,int,double> value = [Device].Move_GetPositionWithTime_32Bit(int axis)		
LabVIEW		
getPositionWithTime_32Bit.vi		



moveReference

This function starts an approach to the reference position. A running motion command is aborted; closed loop moving is switched on. Requires a valid reference position.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.move.moveReference		
params: [axis]		
Result: [errNo]		
C-DLL call		
int AMC_move_moveReference (int deviceHandle, int axis)		
Python		
[dev].move.moveReference(axis)		
Matlab		
[] = move_moveReference(axis)		
C#		
void value = [Device].Move_MoveReference(int axis)		
LabVIEW		
moveReference.vi		

performNSteps

Perform the OL command for N steps

Function specific parameters		
In	axis	[0 1 2]
	backward	Selects the desired direction. False triggers a forward step, true a backward step
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.move.performNSteps		
params: [axis, backward]		
Result: [errNo]		
C-DLL call		
int AMC_move_performNSteps (int deviceHandle, int axis, bool backward)		
Python		
[dev].move.performNSteps (axis, backward)		
Matlab		
[] = move_performNSteps (axis, backward)		
C#		
void value = [Device]. Move_PerformNSteps (int axis, bool backward)		
LabVIEW		
performNSteps.vi		



setControlContinuousBkwd

This function sets a continuous movement on the selected axis in backward direction.

Function specific parameters		
In	axis	[0 1 2]
	enable	If enabled a present movement in the opposite direction is stopped. The parameter "false" stops all movement of the axis regardless its direction
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.move.setControlContinuousBkwd		
params: [axis, enable]		
Result: [errNo]		
C-DLL call		
int AMC_move_setControlContinuousBkwd (int deviceHandle, int axis, bool enable)		
Python		
[dev].move.setControlContinuousBkwd (axis, enable)		
Matlab		
[] = move_setControlContinuousBkwd (axis, enable)		
C#		
void value = [Device]. Move_SetControlContinuousBkwd (int axis, bool enable)		
LabVIEW		
setControlContinuousBkwd.vi		

setControlContinuousFwd

This function sets a continuous movement on the selected axis in positive direction.

Function specific parameters		
In	axis	[0 1 2]
	enable	If enabled a present movement in the opposite direction is stopped. The parameter "false" stops all movement of the axis regardless its direction.
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.move.setControlContinuousFwd		
params: [axis, enable]		
Result: [errNo]		
C-DLL call		
int AMC_move_setControlContinuousFwd (int deviceHandle, int axis, bool enable)		
Python		
[dev].move.setControlContinuousFwd (axis, enable)		
Matlab		
[] = move_setControlContinuousFwd (axis, enable)		
C#		
void value = [Device]. Move_SetControlContinuousFwd (int axis, bool enable)		
LabVIEW		
setControlContinuousFwd.vi		



setControlEotOutputDeactive

This function sets the output applied to the selected axis on the end of travel.

Function specific parameters		
In	axis	[0 1 2]
	enable	if enabled, the output of the axis will be deactivated on positive EOT detection.
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.move.setControlEotOutputDeactive		
params: [axis, enable]		
Result: [errNo]		
C-DLL call		
int AMC_move_setControlEotOutputDeactive (int deviceHandle, int axis, bool enable)		
Python		
[dev].move.setControlEotOutputDeactive (axis, enable)		
Matlab		
[] = move_setControlEotOutputDeactive (axis, enable)		
C#		
void value = [Device]. Move_SetControlEotOutputDeactive (int axis, bool enable)		
LabVIEW		
setControlEotOutputDeactive.vi		

setControlTargetPosition

This function sets the target position for the movement on the selected axis.

The maximum position is +/-10'000'000'000 nm or uDeg, but please make sure not to enter a target position outside of your positioner's range.

Function specific parameters		
In	axis	[0 1 2]
	target	position: in nm for linear actuators for goniometers and rotators the unit is udeg.
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.move.setControlTargetPosition		
params: [axis, target]		
Result: [errNo]		
C-DLL call		
int AMC_move_setControlTargetPosition (int deviceHandle, int axis, double target)		
Python		
[dev].move.setControlTargetPosition (axis, target)		
Matlab		
[] = move_setControlTargetPosition (axis, target)		
C#		
void value = [Device]. Move_SetControlTargetPosition (int axis, double target)		
LabVIEW		
setControlTargetPosition.vi		



setGroundAxis

Pull axis piezo drive to GND actively.

Please note, that this function can only be used on an active axis (see setControlOutput)

This function is only available on AMC300 devices.

Function specific parameters		
In	axis	motion controller axis [0 1 2]
	enabled	true or false
Out	errNo	0 or error
JSON Method		
method: com.attocube.amc.move.setGroundAxis		
params: [axis, enabled]		
Result: [errNo]		
C-DLL call		
int AMC_move_setGroundAxis (int deviceHandle, int axis, bool enabled)		
Python		
[dev].move.setGroundAxis (axis, enabled)		
Matlab		
[] = move_setGroundAxis (axis, enabled)		
C#		
void value = [Device]. Move_SetGroundAxis (int axis, bool enabled)		
LabVIEW		
setGroundAxis.vi		

setGroundAxisAutoOnTarget

Pull axis piezo drive to GND actively if positioner is in ground target range

only in AMC300

this is used in MIC-Mode

Function specific parameters		
In	axis	motion controller axis [0 1 2]
	enabled	true or false
Out	errNo	0 or error
JSON Method		
method: com.attocube.amc.move.setGroundAxisAutoOnTarget		
params: [axis, enabled]		
Result: [errNo]		
C-DLL call		
int AMC_move_setGroundAxisAutoOnTarget (int deviceHandle, int axis, bool enabled)		
Python		
[dev].move.setGroundAxisAutoOnTarget (axis, enabled)		
Matlab		
[] = move_setGroundAxisAutoOnTarget (axis, enabled)		
C#		
void value = [Device]. Move_SetGroundAxisAutoOnTarget (int axis, bool enabled)		
LabVIEW		
setGroundAxisAutoOnTarget.vi		



setGroundTargetRange

Set the range around the target position in which the auto grounding becomes active.

only in AMC300

Function specific parameters		
In	axis	[0 1 2]
	range	in nm
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.move.setGroundTargetRange		
params: [axis, range]		
Result: [errNo]		
C-DLL call		
int AMC_move_setGroundTargetRange (int deviceHandle, int axis, int range)		
Python		
[dev].move.setGroundTargetRange (axis, range)		
Matlab		
[] = move_setGroundTargetRange (axis, range)		
C#		
void value = [Device]. Move_SetGroundTargetRange (int axis, int range)		
LabVIEW		
setGroundTargetRange.vi		

setNSteps

This function triggers n steps on the selected axis in desired direction. /PRO feature.

Function specific parameters		
In	axis	[0 1 2]
	backward	Selects the desired direction. False triggers a forward step, true a backward step
	step	number of step
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.move.setNSteps		
params: [axis, backward, step]		
Result: [errNo]		
C-DLL call		
int AMC_move_setNSteps (int deviceHandle, int axis, bool backward, int step)		
Python		
[dev].move.setNSteps (axis, backward, step)		
Matlab		
[] = move_setNSteps (axis, backward, step)		
C#		
void value = [Device]. Move_SetNSteps (int axis, bool backward, int step)		
LabVIEW		
setNSteps.vi		



setSingleStep

This function triggers one step on the selected axis in desired direction.

Function specific parameters		
In	axis	[0 1 2]
	backward	Selects the desired direction. False triggers a forward step, true a backward step
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.move.setSingleStep		
params: [axis, backward]		
Result: [errNo]		
C-DLL call		
int AMC_move_setSingleStep (int deviceHandle, int axis, bool backward)		
Python		
[dev].move.setSingleStep (axis, backward)		
Matlab		
[] = move_setSingleStep (axis, backward)		
C#		
void value = [Device]. Move_SetSingleStep (int axis, bool backward)		
LabVIEW		
setSingleStep.vi		

writeNSteps

Sets the number of steps to perform on stepwise movement. /PRO feature.

Function specific parameters		
In	axis	[0 1 2]
	step	number of step
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.move.writeNSteps		
params: [axis, step]		
Result: [errNo]		
C-DLL call		
int AMC_move_writeNSteps (int deviceHandle, int axis, int step)		
Python		
[dev].move.writeNSteps (axis, step)		
Matlab		
[] = move_writeNSteps (axis, step)		
C#		
void value = [Device]. Move_WriteNSteps (int axis, int step)		
LabVIEW		
writeNSteps.vi		



3.8. Network

apply

Applies the temporary IP configuration and loads it.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.apply		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_network_apply (int deviceHandle)		
Python		
[dev].network.apply()		
Matlab		
[] = network_apply()		
C#		
void value = [Device]. Network_Apply()		
LabVIEW		
apply.vi		

configureWifi

Changes the WiFi configuration and applies it.

Function specific parameters		
In	mode	0 = access point; 1 = WiFi client
	SSID	SSID string
	psk	pre-shared key
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.configureWifi		
params: [mode, SSID, psk]		
Result: [errNo]		
C-DLL call		
int AMC_network_configureWifi (int deviceHandle, int mode, const char* SSID, const char* psk)		
Python		
[dev].network.configureWifi (mode, SSID, psk)		
Matlab		
[] = network_configureWifi (mode, SSID, psk)		
C#		
void value = [Device]. Network_ConfigureWifi (int mode, string SSID, string psk)		
LabVIEW		
configureWifi.vi		



discard

Discards the temporary IP configuration.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.discard		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_network_discard (int deviceHandle)		
Python		
[dev].network.discard()		
Matlab		
[] = network_discard()		
C#		
void value = [Device].Network_Discard()		
LabVIEW		
discard.vi		

getDefaultGateway

Gets the default gateway of the device.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	gateway	default gateway as string
JSON Method		
method: com.attocube.system.network.getDefaultGateway		
params: []		
Result: [errNo, gateway]		
C-DLL call		
int AMC_network_getDefaultGateway (int deviceHandle, char* gateway, int size0)		
Python		
gateway = [dev].network.getDefaultGateway()		
Matlab		
[gateway] = network_getDefaultGateway ()		
C#		
string value = [Device]. Network_GetDefaultGateway ()		
LabVIEW		
getDefaultGateway.vi		



getDnsResolver

Gets the DNS resolver.

Function specific parameters		
In	priority	priority of DNS resolver (0 = default; 1 = backup)
Out	errNo	error code, if there was an error, otherwise 0 for ok
	resolver	IP address of DNS resolver
JSON Method		
method: com.attocube.system.network.getDnsResolver		
params: [priority]		
Result: [errNo, resolver]		
C-DLL call		
int AMC_network_getDnsResolver (int deviceHandle, int priority, char* resolver, int size0)		
Python		
resolver = [dev].network.getDnsResolver (priority)		
Matlab		
[resolver] = network_getDnsResolver (priority)		
C#		
string value = [Device]. Network_GetDnsResolver (int priority)		
LabVIEW		
getDnsResolver.vi		

getEnableDhcpClient

Gets the status of DHCP client.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	enabled	true = enabled; false = disabled
JSON Method		
method: com.attocube.system.network.getEnableDhcpClient		
params: []		
Result: [errNo, enabled]		
C-DLL call		
int AMC_network_getEnableDhcpClient (int deviceHandle, bool* enabled)		
Python		
enabled = [dev].network.getEnableDhcpClient()		
Matlab		
[enabled] = network_getEnableDhcpClient ()		
C#		
bool value = [Device]. Network_GetEnableDhcpClient ()		
LabVIEW		
getEnableDhcpClient.vi		



getEnableDhcpServer

Gets the status of DHCP server.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	enabled	true = enabled; false = disabled
JSON Method		
method: com.attocube.system.network.getEnableDhcpServer		
params: []		
Result: [errNo, enabled]		
C-DLL call		
int AMC_network_getEnableDhcpServer (int deviceHandle, bool* enabled)		
Python		
enabled = [dev].network.getEnableDhcpServer()		
Matlab		
[enabled] = network_getEnableDhcpServer ()		
C#		
bool value = [Device]. Network_GetEnableDhcpServer ()		
LabVIEW		
getEnableDhcpServer.vi		

getIpAddress

Gets the IP address of the device.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	IP	address as string
JSON Method		
method: com.attocube.system.network.getIpAddress		
params: []		
Result: [errNo, IP]		
C-DLL call		
int AMC_network_getIpAddress (int deviceHandle, char* IP, int size0)		
Python		
IP = [dev].network.getIpAddress()		
Matlab		
[IP] = network_getIpAddress ()		
C#		
string value = [Device]. Network_GetIpAddress ()		
LabVIEW		
getIpAddress.vi		



getProxyServer

Gets the proxy settings of the device.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	proxyServer	proxy server setting, empty for no proxy
JSON Method		
method: com.attocube.system.network.getProxyServer		
params: []		
Result: [errNo, proxyServer]		
C-DLL call		
int AMC_network_getProxyServer (int deviceHandle, char* proxyServer, int size0)		
Python		
proxyServer = [dev].network.getProxyServer()		
Matlab		
[proxyServer] = network_getProxyServer ()		
C#		
string value = [Device]. Network_GetProxyServer ()		
LabVIEW		
getProxyServer.vi		

getRealIpAddress

Gets the real IP address of the device set to the network interface (br0, eth1 or eth0).

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	IP	IP address
JSON Method		
method: com.attocube.system.network.getRealIpAddress		
params: []		
Result: [errNo, IP]		
C-DLL call		
int AMC_network_getRealIpAddress (int deviceHandle, char* IP, int size0)		
Python		
IP = [dev].network.getRealIpAddress()		
Matlab		
[IP] = network_getRealIpAddress ()		
C#		
string value = [Device]. Network_GetRealIpAddress ()		
LabVIEW		
getRealIpAddress.vi		



getSubnetMask

Gets the subnet mask of the device.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	netmask	subnet mask as string
JSON Method		
method: com.attocube.system.network.getSubnetMask		
params: []		
Result: [errNo, netmask]		
C-DLL call		
int AMC_network_getSubnetMask (int deviceHandle, char* netmask, int size0)		
Python		
netmask = [dev].network.getSubnetMask()		
Matlab		
[netmask] = network_getSubnetMask ()		
C#		
string value = [Device]. Network_GetSubnetMask ()		
LabVIEW		
getSubnetMask.vi		

getWifiMode

Gets the operation mode of the WiFi adapter.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	mode	0 = access point; 1 = WiFi client
JSON Method		
method: com.attocube.system.network.getWifiMode		
params: []		
Result: [errNo, mode]		
C-DLL call		
int AMC_network_getWifiMode (int deviceHandle, int* mode)		
Python		
mode = [dev].network.getWifiMode()		
Matlab		
[mode] = network_getWifiMode ()		
C#		
int value = [Device]. Network_GetWifiMode ()		
LabVIEW		
getWifiMode.vi		



getWifiPassphrase

Gets the passphrase of the network hosted (mode: Access point) or connected to (mode: client).

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	psk	pre-shared key
JSON Method		
method: com.attocube.system.network.getWifiPassphrase		
params: []		
Result: [errNo, psk]		
C-DLL call		
int AMC_network_getWifiPassphrase (int deviceHandle, char* psk, int size0)		
Python		
psk = [dev].network.getWifiPassphrase()		
Matlab		
[psk] = network_getWifiPassphrase ()		
C#		
string value = [Device]. Network_GetWifiPassphrase ()		
LabVIEW		
getWifiPassphrase.vi		

getWifiPresent

Checks if a WiFi interface is present.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	present	true if the interface is present, else false
JSON Method		
method: com.attocube.system.network.getWifiPresent		
params: []		
Result: [errNo, present]		
C-DLL call		
int AMC_network_getWifiPresent (int deviceHandle, bool* present)		
Python		
present = [dev].network.getWifiPresent()		
Matlab		
[present] = network_getWifiPresent ()		
C#		
bool value = [Device]. Network_GetWifiPresent ()		
LabVIEW		
getWifiPresent.vi		



getWifiSSID

Gets the SSID of the network hosted (mode: access point) or connected to (mode: client).

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	SSID	SSID string
JSON Method		
method: com.attocube.system.network.getWifiSSID		
params: []		
Result: [errNo, SSID]		
C-DLL call		
int AMC_network_getWifiSSID (int deviceHandle, char* SSID, int size0)		
Python		
SSID = [dev].network.getWifiSSID()		
Matlab		
[SSID] = network_getWifiSSID()		
C#		
string value = [Device].Network_GetWifiSSID()		
LabVIEW		
getWifiSSID.vi		

setDefaultGateway

Sets the default gateway of the device.

Function specific parameters		
In	gateway	default gateway as string
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.setDefaultGateway		
params: [gateway]		
Result: [errNo]		
C-DLL call		
int AMC_network_setDefaultGateway (int deviceHandle, const char* gateway)		
Python		
[dev].network.setDefaultGateway(gateway)		
Matlab		
[] = network_setDefaultGateway (gateway)		
C#		
void value = [Device]. Network_SetDefaultGateway (string gateway)		
LabVIEW		
setDefaultGateway.vi		



setDnsResolver

Sets the DNS resolver.

Function specific parameters		
In	priority	priority of DNS resolver (0 = default; 1 = backup)
	resolver	IP address of DNS resolver
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.setDnsResolver		
params: [priority, resolver]		
Result: [errNo]		
C-DLL call		
int AMC_network_setDnsResolver (int deviceHandle, int priority, const char* resolver)		
Python		
[dev].network.setDnsResolver (priority, resolver)		
Matlab		
[] = network_setDnsResolver (priority, resolver)		
C#		
void value = [Device]. Network_SetDnsResolver (int priority, string resolver)		
LabVIEW		
setDnsResolver.vi		

setEnabledDhcpClient

Enables or disables DHCP client.

Function specific parameters		
In	enable	true = enable; false = disable
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.setEnabledDhcpClient		
params: [enable]		
Result: [errNo]		
C-DLL call		
int AMC_network_setEnabledDhcpClient (int deviceHandle, bool enable)		
Python		
[dev].network.setEnabledDhcpClient(enable)		
Matlab		
[] = network_setEnabledDhcpClient (enable)		
C#		
void value = [Device]. Network_SetEnabledDhcpClient (bool enable)		
LabVIEW		
setEnabledDhcpClient.vi		



setEnableDhcpServer

Enables or disables DHCP server.

Function specific parameters		
In	enable	true = enable; false = disable
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.setEnableDhcpServer		
params: [enable]		
Result: [errNo]		
C-DLL call		
int AMC_network_setEnableDhcpServer (int deviceHandle, bool enable)		
Python		
[dev].network.setEnableDhcpServer(enable)		
Matlab		
[] = network_setEnableDhcpServer (enable)		
C#		
void value = [Device]. Network_SetEnableDhcpServer (bool enable)		
LabVIEW		
setEnableDhcpServer.vi		

setIpAddress

Sets the IP address of the device.

Function specific parameters		
In	address	IP address as string
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.setIpAddress		
params: [address]		
Result: [errNo]		
C-DLL call		
int AMC_network_setIpAddress (int deviceHandle, const char* address)		
Python		
[dev].network.setIpAddress(address)		
Matlab		
[] = network_setIpAddress (address)		
C#		
void value = [Device]. Network_SetIpAddress (string address)		
LabVIEW		
setIpAddress.vi		



setProxyServer

Sets the proxy server of the device.

Function specific parameters		
In	proxyServer	proxy server setting
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.setProxyServer		
params: [proxyServer]		
Result: [errNo]		
C-DLL call		
int AMC_network_setProxyServer (int deviceHandle, const char* proxyServer)		
Python		
[dev].network.setProxyServer(proxyServer)		
Matlab		
[] = network_setProxyServer (proxyServer)		
C#		
void value = [Device]. Network_SetProxyServer (string proxyServer)		
LabVIEW		
setProxyServer.vi		

setSubnetMask

Sets the subnet mask of the device.

Function specific parameters		
In	netmask	subnet mask as string
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.setSubnetMask		
params: [netmask]		
Result: [errNo]		
C-DLL call		
int AMC_network_setSubnetMask (int deviceHandle, const char* netmask)		
Python		
[dev].network.setSubnetMask(netmask)		
Matlab		
[] = network_setSubnetMask (netmask)		
C#		
void value = [Device]. Network_SetSubnetMask (string netmask)		
LabVIEW		
setSubnetMask.vi		



setWifiMode

Changes the operation mode of the WiFi adapter.

Function specific parameters		
In	mode	0 = access point; 1 = WiFi client
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.setWifiMode		
params: [mode]		
Result: [errNo]		
C-DLL call		
int AMC_network_setWifiMode (int deviceHandle, int mode)		
Python		
[dev].network.setWifiMode(mode)		
Matlab		
[] = network_setWifiMode (mode)		
C#		
void value = [Device]. Network_SetWifiMode (int mode)		
LabVIEW		
setWifiMode.vi		

setWifiPassphrase

Changes the passphrase of the network hosted (mode: access point) or connected to (mode: client).

Function specific parameters		
In	psk	pre-shared key
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.setWifiPassphrase		
params: [psk]		
Result: [errNo]		
C-DLL call		
int AMC_network_setWifiPassphrase (int deviceHandle, const char* psk)		
Python		
[dev].network.setWifiPassphrase(psk)		
Matlab		
[] = network_setWifiPassphrase (psk)		
C#		
void value = [Device]. Network_SetWifiPassphrase (string psk)		
LabVIEW		
setWifiPassphrase.vi		



setWifiSSID

Changes the SSID of the network hosted (mode: access point) or connected to (mode: client).

Function specific parameters		
In	SSID	SSID string
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.setWifiSSID		
params: [SSID]		
Result: [errNo]		
C-DLL call		
int AMC_network_setWifiSSID (int deviceHandle, const char* SSID)		
Python		
[dev].network.setWifiSSID(SSID)		
Matlab		
[] = network_setWifiSSID (SSID)		
C#		
void value = [Device]. Network_SetWifiSSID (string SSID)		
LabVIEW		
setWifiSSID.vi		

verify

Verifies if the temporary IP configuration is correct.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.network.verify		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_network_verify (int deviceHandle)		
Python		
[dev].network.verify()		
Matlab		
[] = network_verify()		
C#		
void value = [Device].Network_Verify()		
LabVIEW		
verify.vi		



3.9. Res

getChainGain

Get chain gain, see setChainGain for parameter description

Function specific parameters		
In	axis	number of axis
Out	errNo	errNo
	gaincoeff	or voltage in mV for dcres
JSON Method		
method: com.attocube.amc.res.getChainGain		
params: [axis]		
Result: [errNo, gaincoeff]		
C-DLL call		
int AMC_res_getChainGain (int deviceHandle, int axis, int* gaincoeff)		
Python		
gaincoeff = [dev].res.getChainGain(axis)		
Matlab		
[gaincoeff] = res_getChainGain (axis)		
C#		
int value = [Device]. Res_GetChainGain (int axis)		
LabVIEW		
getChainGain.vi		

getLinearization

Gets whether linearization is enabled or not

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enabled	true when enabled
JSON Method		
method: com.attocube.amc.res.getLinearization		
params: [axis]		
Result: [errNo, enabled]		
C-DLL call		
int AMC_res_getLinearization (int deviceHandle, int axis, bool* enabled)		
Python		
enabled = [dev].res.getLinearization (axis)		
Matlab		
[enabled] = res_getLinearization (axis)		
C#		
bool value = [Device]. Res_GetLinearization (int axis)		
LabVIEW		
getLinearization.vi		



getLutSn

get the identifier of the loaded lookuptable (will be empty if disabled)

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	value_string1	string : identifier
JSON Method		
method: com.attocube.amc.res.getLutSn		
params: [axis]		
Result: [errNo, value_string1]		
C-DLL call		
int AMC_res_getLutSn (int deviceHandle, int axis, char* value_string1, int size0)		
Python		
value_string1 = [dev].res.getLutSn (axis)		
Matlab		
[value_string1] = res_getLutSn (axis)		
C#		
string value = [Device]. Res_GetLutSn (int axis)		
LabVIEW		
getLutSn.vi		

getMode

Get mode of RES application, see setMode for the description of possible parameters

Function specific parameters		
Out	errNo	errNo
	mode	mode
JSON Method		
method: com.attocube.amc.res.getMode		
params: []		
Result: [errNo, mode]		
C-DLL call		
int AMC_res_getMode (int deviceHandle, int* mode)		
Python		
mode = [dev].res.getMode()		
Matlab		
[mode] = res_getMode()		
C#		
int value = [Device].Res_GetMode()		
LabVIEW		
getMode.vi		



getResFilterSetting

This function retrieves the current setting of the RES-signal filter.

Function specific parameters		
Out	err	error number
		res_filter_cfg_nbr number of the filter setting minimal = 1, low = 2, balanced = 3, high = 4, maximal = 5,
JSON Method		
method: com.attocube.amc.res.getResFilterSetting		
params: []		
Result: [err,]		
C-DLL call		
int AMC_res_getResFilterSetting (int deviceHandle, int* err, int*)		
Python		
err, = [dev].res.getResFilterSetting()		
Matlab		
[err,] = res_getResFilterSetting ()		
C#		
Tuple<int,int> value = [Device]. Res_GetResFilterSetting ()		
LabVIEW		
getResFilterSetting.vi		

getSensorStatus

Gets whether a valid RES position signal is present (always true for a disabled sensor and for rotators)

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	present	true when present
JSON Method		
method: com.attocube.amc.res.getSensorStatus		
params: [axis]		
Result: [errNo, present]		
C-DLL call		
int AMC_res_getSensorStatus (int deviceHandle, int axis, bool* present)		
Python		
present = [dev].res.getSensorStatus(axis)		
Matlab		
[present] = res_getSensorStatus (axis)		
C#		
bool value = [Device]. Res_GetSensorStatus (int axis)		
LabVIEW		
getSensorStatus.vi		



setChainGain

Set signal chain gain to control overall power

Function specific parameters		
In	axis	number of axis
	gainconfig	0: 0dB (power $600\text{mVpkpk}^2/R$), 1 : -10 dB , 2 : -15 dB , 3 : -20 dB
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.res.setChainGain		
params: [axis, gainconfig]		
Result: [errNo]		
C-DLL call		
int AMC_res_setChainGain (int deviceHandle, int axis, int gainconfig)		
Python		
[dev].res.setChainGain (axis, gainconfig)		
Matlab		
[] = res_setChainGain (axis, gainconfig)		
C#		
void value = [Device]. Res_SetChainGain (int axis, int gainconfig)		
LabVIEW		
setChainGain.vi		

setConfigurationFile

Load configuration file which either contains a JSON dict with parameters for the positioner on the axis or the LUT file itself (as legacy support for ANC350 .aps files)

Function specific parameters		
In	axis	[0 1 2]
	content	JSON Dictionary or .aps File. The JSON Dictionary can/must contain the following keys: 'type': mandatory This field has to be one of the positioner list (see getPositionersList) 'lut': optional, contains an array of 1024 LUT values that are a mapping between ratio of the RES element travelled (0 to 1) and the corresponding absolute value at this ratio given in [nm]. Note: when generating these tables with position data in absolute units, the scaling of the travel ratio with the current sensor range has to be reversed. 'lut_sn': optional, a string to uniquely identify the loaded LUT
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.res.setConfigurationFile		
params: [axis, content]		
Result: [errNo]		
C-DLL call		
int AMC_res_setConfigurationFile (int deviceHandle, int axis, const char* content)		
Python		
[dev].res.setConfigurationFile(axis, content)		
Matlab		
[] = res_setConfigurationFile(axis, content)		
C#		
void value = [Device].Res_SetConfigurationFile(int axis, string content)		
LabVIEW		
setConfigurationFile.vi		



setLinearization

Control if linearization is enabled or not.

Please note, that this using the linearization can only be enable, if a LUT-file has been uploaded (see setLinTable)

Please note, that active closed loop movement will be stopped when running this function to avoid unexpected movements due to the position change.

Function specific parameters		
In	axis	[0 1 2]
	enable	boolean (true: enable linearization)
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.res.setLinearization		
params: [axis, enable]		
Result: [errNo]		
C-DLL call		
int AMC_res_setLinearization (int deviceHandle, int axis, bool enable)		
Python		
[dev].res.setLinearization (axis, enable)		
Matlab		
[] = res_setLinearization (axis, enable)		
C#		
void value = [Device]. Res_SetLinearization (int axis, bool enable)		
LabVIEW		
setLinearization.vi		

setMode

Sets the mode of the RES position measurement

This selects which frequency/ies are used for the lock-in measurement in AC-mode and whether or not common wires are shared

1: (ACRES)Individual per axis: each axis is measured on a different frequency; this mode reduces noise coupling between axes, while requiring more wiring

2: (ACRES)Shared line/MIC-Mode: each axis is measured on the same frequency, which reduces the number of required wires while more coupling noise is expected

3: (ACRES)Same as 1, but with overall lower frequencies. This reduces the influence on the measurement accuracy on longer cables

4: (ACRES)Same as 2, but with overall lower frequencies. This reduces the influence on the measurement accuracy on longer cables

5: (DCRES)Individual per axis: the RES-sensor of each axis has its own voltage supply and ground wire -> less coupling noise but more wires

6: (DCRES)Shared: the RES-sensors of all axes share supply- and ground-wire -> more coupling noise, but less wires

Note, that the allowed modes depend on the read-out-type of the RES-AMC. For ACRES only 1-4 and for DCRES only 5 and 6 are allowed modes.

Function specific parameters		
In	mode	1: Individual per axis, 2: Shared line mode, 3: Individual per axis (low frequency), 4: Shared line mode (low frequency), 5: Individual per axis (DCRES), 6: Shared line mode (DCRES)
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.res.setMode		
params: [mode]		
Result: [errNo]		
C-DLL call		
int AMC_res_setMode (int deviceHandle, int mode)		
Python		
[dev].res.setMode(mode)		
Matlab		
[] = res_setMode(mode)		
C#		
void value = [Device].Res_SetMode(int mode)		
LabVIEW		
setMode.vi		



setResFilterSetting

This function sets the RES-signal filters according to the passed config-number;

Note, that the sensor-readout will be interrupted for a short time while this function is executed;

it is not permitted to run this function while there is an active closed loop movement on any axis;

If you want to keep the filter settings from previous fw-versions, please use 'balanced';

The default setting is 'balanced'.

Function specific parameters		
In	res_filter_cfg_nbr	: number of the desired filter-setting minimal = 1, low = 2, balanced = 3, high = 4, maximal = 5,
Out	err	error-number
JSON Method		
method: com.attocube.amc.res.setResFilterSetting		
params: [res_filter_cfg_nbr]		
Result: [err]		
C-DLL call		
int AMC_res_setResFilterSetting (int deviceHandle, int res_filter_cfg_nbr, int* err)		
Python		
err = [dev].res.setResFilterSetting(res_filter_cfg_nbr)		
Matlab		
[err] = res_setResFilterSetting (res_filter_cfg_nbr)		
C#		
int value = [Device]. Res_SetResFilterSetting (int res_filter_cfg_nbr)		
LabVIEW		
setResFilterSetting.vi		

3.10. Rotcomp

getControlTargetRanges

Checks if all three axis are in target range.

Function specific parameters		
Out	errNo	Error code, if there was an error, otherwise 0 for ok
	in_target_range	true all three axes are in target range, false at least one axis is not in target range
JSON Method		
method: com.attocube.amc.rotcomp.getControlTargetRanges		
params: []		
Result: [errNo, in_target_range]		
C-DLL call		
int AMC_rotcomp_getControlTargetRanges (int deviceHandle, bool* in_target_range)		
Python		
in_target_range = [dev].rotcomp.getControlTargetRanges()		
Matlab		
[in_target_range] = rotcomp_getControlTargetRanges()		
C#		
bool value = [Device].Rotcomp_GetControlTargetRanges()		
LabVIEW		
getControlTargetRanges.vi		



getEnabled

Gets the enabled status of the rotation compensation

Function specific parameters		
Out	errNo	Error code, if there was an error, otherwise 0 for ok
	enabled	true Rotation compensation is enabled, false Rotation compensation is disabled
JSON Method		
method: com.attocube.amc.rotcomp.getEnabled		
params: []		
Result: [errNo, enabled]		
C-DLL call		
int AMC_rotcomp_getEnabled (int deviceHandle, bool* enabled)		
Python		
enabled = [dev].rotcomp.getEnabled()		
Matlab		
[enabled] = rotcomp_getEnabled()		
C#		
bool value = [Device].Rotcomp_GetEnabled()		
LabVIEW		
getEnabled.vi		

getLUT

Gets the LUT file as JSON string

Function specific parameters		
Out	errNo	Error code, if there was an error, otherwise 0 for ok
	lut	JSON string of the LUT file for the rotation compensation
JSON Method		
method: com.attocube.amc.rotcomp.getLUT		
params: []		
Result: [errNo, lut]		
C-DLL call		
int AMC_rotcomp_getLUT (int deviceHandle, char* lut, int size0)		
Python		
lut = [dev].rotcomp.getLUT()		
Matlab		
[lut] = rotcomp_getLUT()		
C#		
string value = [Device].Rotcomp_GetLUT()		
LabVIEW		
getLUT.vi		



setEnabled

Enables and disables the rotation compensation

Function specific parameters		
In	enabled	true Rotation compensation is enabled, false Rotation compensation is disabled
Out	errNo	Error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.amc.rotcomp.setEnabled		
params: [enabled]		
Result: [errNo]		
C-DLL call		
int AMC_rotcomp_setEnabled (int deviceHandle, bool enabled)		
Python		
[dev].rotcomp.setEnabled(enabled)		
Matlab		
[] = rotcomp_setEnabled (enabled)		
C#		
void value = [Device]. Rotcomp_SetEnabled (bool enabled)		
LabVIEW		
setEnabled.vi		

setLUT

Sets the LUT file from a JSON string

Function specific parameters		
In	lut_string	JSON string of the LUT file for the rotation compensation
Out	errNo	Error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.amc.rotcomp.setLUT		
params: [lut_string]		
Result: [errNo]		
C-DLL call		
int AMC_rotcomp_setLUT (int deviceHandle, const char* lut_string)		
Python		
[dev].rotcomp.setLUT(lut_string)		
Matlab		
[] = rotcomp_setLUT (lut_string)		
C#		
void value = [Device]. Rotcomp_SetLUT (string lut_string)		
LabVIEW		
setLUT.vi		



updateOffsets

Updates the start offsets of the axes

Function specific parameters		
In	offset_axis0	Offset of axis 1 in [nm]
	offset_axis1	Offset of axis 2 in [nm]
	offset_axis2	Offset of axis 3 in [nm]
Out	errNo	Error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.amc.rotcomp.updateOffsets		
params: [offset_axis0, offset_axis1, offset_axis2]		
Result: [errNo]		
C-DLL call		
int AMC_rotcomp_updateOffsets (int deviceHandle, int offset_axis0, int offset_axis1, int offset_axis2)		
Python		
[dev].rotcomp.updateOffsets (offset_axis0, offset_axis1, offset_axis2)		
Matlab		
[] = rotcomp_updateOffsets (offset_axis0, offset_axis1, offset_axis2)		
C#		
void value = [Device]. Rotcomp_UpdateOffsets (int offset_axis0, int offset_axis1, int offset_axis2)		
LabVIEW		
updateOffsets.vi		

3.11. Rtin

apply

Apply all realtime input function

Function specific parameters		
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtin.apply		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_rtin_apply (int deviceHandle)		
Python		
[dev].rtin.apply()		
Matlab		
[] = rtin_apply()		
C#		
void value = [Device]. Rtin_Apply ()		
LabVIEW		
rtin_apply.vi		



discard

Discard all values beting set and not yet applied

Function specific parameters		
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtin.discard		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_rtin_discard (int deviceHandle)		
Python		
[dev].rtin.discard()		
Matlab		
[] = rtin_discard()		
C#		
void value = [Device].Rtin_Discard()		
LabVIEW		
rtin_discard.vi		

getControlAQuadBInResolution

This function gets the AQuadB input resolution for setpoint parameter.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	resolution	ion nm
JSON Method		
method: com.attocube.amc.rtin.getControlAQuadBInResolution		
params: [axis]		
Result: [errNo, resolution]		
C-DLL call		
int AMC_rtin_getControlAQuadBInResolution (int deviceHandle, int axis, int* resolution)		
Python		
resolution = [dev].rtin.getControlAQuadBInResolution(axis)		
Matlab		
[resolution] = rtin_getControlAQuadBInResolution (axis)		
C#		
int value = [Device]. Rtin_GetControlAQuadBInResolution (int axis)		
LabVIEW		
rtin_getControlAQuadBInResolution.vi		



getControlMoveGPIO

This function gets the status for real time input on the selected axis in closed-loop mode.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enable	boolean true: approach enabled , false: approach disabled
JSON Method		
method: com.attocube.amc.rtin.getControlMoveGPIO		
params: [axis]		
Result: [errNo, enable]		
C-DLL call		
int AMC_rtin_getControlMoveGPIO (int deviceHandle, int axis, bool* enable)		
Python		
enable = [dev].rtin.getControlMoveGPIO(axis)		
Matlab		
[enable] = rtin_getControlMoveGPIO(axis)		
C#		
bool value = [Device].Rtin_GetControlMoveGPIO(int axis)		
LabVIEW		
rtin_getControlMoveGPIO.vi		

getGpioMode

get the GPIO mode for Mic Mode feature

Function specific parameters		
Out	errNo	errNo
	gpio_mode	gpio_mode: 0: Standard GPIO 1: NSL-/Mic-Mode
JSON Method		
method: com.attocube.amc.rtin.getGpioMode		
params: []		
Result: [errNo, gpio_mode]		
C-DLL call		
int AMC_rtin_getGpioMode (int deviceHandle, int* gpio_mode)		
Python		
gpio_mode = [dev].rtin.getGpioMode()		
Matlab		
[gpio_mode] = rtin_getGpioMode()		
C#		
int value = [Device].Rtin_GetGpioMode()		
LabVIEW		
rtin_getGpioMode.vi		



getNsIMux

get the axis the NSL multiplexer is set to

Function specific parameters		
Out	errNo	errNo
	mux_mode	0: Off 1: Axis 1 2: Axis 2 3: Axis 3
JSON Method		
method: com.attocube.amc.rtin.getNsIMux		
params: []		
Result: [errNo, mux_mode]		
C-DLL call		
int AMC_rtin_getNsIMux (int deviceHandle, int* mux_mode)		
Python		
mux_mode = [dev].rtin.getNsIMux()		
Matlab		
[mux_mode] = rtin_getNsIMux()		
C#		
int value = [Device].Rtin_GetNsIMux()		
LabVIEW		
rtin_getNsIMux.vi		

getRealTimeInChangePerPulse

This function gets the change per pulse for the selected axis under real time input in the closed-loop mode.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	resolution	to be added in current pos in nm
JSON Method		
method: com.attocube.amc.rtin.getRealTimeInChangePerPulse		
params: [axis]		
Result: [errNo, resolution]		
C-DLL call		
int AMC_rtin_getRealTimeInChangePerPulse (int deviceHandle, int axis, int* resolution)		
Python		
resolution = [dev].rtin.getRealTimeInChangePerPulse (axis)		
Matlab		
[resolution] = rtin_getRealTimeInChangePerPulse (axis)		
C#		
int value = [Device].Rtin_GetRealTimeInChangePerPulse (int axis)		
LabVIEW		
rtin_getRealTimeInChangePerPulse.vi		



getRealTimeInFeedbackLoopMode

Get if the realtime function must operate in close loop operation or open loop operation

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	mode	0: open loop, 1 : close-loop
JSON Method		
method: com.attocube.amc.rtin.getRealTimeInFeedbackLoopMode		
params: [axis]		
Result: [errNo, mode]		
C-DLL call		
int AMC_rtin_getRealTimeInFeedbackLoopMode (int deviceHandle, int axis, int* mode)		
Python		
mode = [dev].rtin.getRealTimeInFeedbackLoopMode(axis)		
Matlab		
[mode] = rtin_getRealTimeInFeedbackLoopMode(axis)		
C#		
int value = [Device].Rtin_GetRealTimeInFeedbackLoopMode(int axis)		
LabVIEW		
rtin_getRealTimeInFeedbackLoopMode.vi		

getRealTimeInMode

This function sets or gets the real time input mode for the selected axis.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	RT_IN_MODE	Please refer to "setRealTimeInMode" for information on possible values
JSON Method		
method: com.attocube.amc.rtin.getRealTimeInMode		
params: [axis]		
Result: [errNo, RT_IN_MODE]		
C-DLL call		
int AMC_rtin_getRealTimeInMode (int deviceHandle, int axis, int* RT_IN_MODE)		
Python		
RT_IN_MODE = [dev].rtin.getRealTimeInMode(axis)		
Matlab		
[RT_IN_MODE] = rtin_getRealTimeInMode(axis)		
C#		
int value = [Device].Rtin_GetRealTimeInMode(int axis)		
LabVIEW		
rtin_getRealTimeInMode.vi		



getRealTimeInStepsPerPulse

Get the change in step per pulse of the realtime input when trigger and stepper mode is used

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	steps	number of steps to applied
JSON Method		
method: com.attocube.amc.rtin.getRealTimeInStepsPerPulse		
params: [axis]		
Result: [errNo, steps]		
C-DLL call		
int AMC_rtin_getRealTimeInStepsPerPulse (int deviceHandle, int axis, int* steps)		
Python		
steps = [dev].rtin.getRealTimeInStepsPerPulse(axis)		
Matlab		
[steps] = rtin_getRealTimeInStepsPerPulse (axis)		
C#		
int value = [Device]. Rtin_GetRealTimeInStepsPerPulse (int axis)		
LabVIEW		
rtin_getRealTimeInStepsPerPulse.vi		

setControlAQuadBInResolution

This function sets the AQuadB input resolution for setpoint parameter.

Function specific parameters		
In	axis	[0 1 2]
	resolution	ion nm
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtin.setControlAQuadBInResolution		
params: [axis, resolution]		
Result: [errNo]		
C-DLL call		
int AMC_rtin_setControlAQuadBInResolution (int deviceHandle, int axis, int resolution)		
Python		
[dev].rtin.setControlAQuadBInResolution (axis, resolution)		
Matlab		
[] = rtin_setControlAQuadBInResolution (axis, resolution)		
C#		
void value = [Device]. Rtin_SetControlAQuadBInResolution (int axis, int resolution)		
LabVIEW		
rtin_setControlAQuadBInResolution.vi		



setControlMoveGPIO

This function sets the status for real time input on the selected axis in closed-loop mode.

Function specific parameters		
In	axis	[0 1 2]
	enable	boolean true: enable the approach , false: disable the approach
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtin.setControlMoveGPIO		
params: [axis, enable]		
Result: [errNo]		
C-DLL call		
int AMC_rtin_setControlMoveGPIO (int deviceHandle, int axis, bool enable)		
Python		
[dev].rtin.setControlMoveGPIO(axis, enable)		
Matlab		
[] = rtin_setControlMoveGPIO(axis, enable)		
C#		
void value = [Device].Rtin_SetControlMoveGPIO(int axis, bool enable)		
LabVIEW		
rtin_setControlMoveGPIO.vi		

setGpioMode

set the GPIO mode for Mic Mode feature

Function specific parameters		
In	gpio_mode	[0 1] 0: Standard GPIO 1: NSL-/Mic-Mode
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtin.setGpioMode		
params: [gpio_mode]		
Result: [errNo]		
C-DLL call		
int AMC_rtin_setGpioMode (int deviceHandle, int gpio_mode)		
Python		
[dev].rtin.setGpioMode(gpio_mode)		
Matlab		
[] = rtin_setGpioMode(gpio_mode)		
C#		
void value = [Device].Rtin_SetGpioMode(int gpio_mode)		
LabVIEW		
rtin_setGpioMode.vi		



setNslMux

set the axis the NSL multiplexer is set to

Function specific parameters		
Ξ	mux_mode	[0 1 2 3] 0: Off 1: Axis 1 2: Axis 2 3: Axis 3
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtin.setNslMux		
params: [mux_mode]		
Result: [errNo]		
C-DLL call		
int AMC_rtin_setNslMux (int deviceHandle, int mux_mode)		
Python		
[dev].rtin.setNslMux (mux_mode)		
Matlab		
[] = rtin_setNslMux (mux_mode)		
C#		
void value = [Device]. Rtin_SetNslMux (int mux_mode)		
LabVIEW		
rtin_setNslMux.vi		

setRealTimeInChangePerPulse

This function sets the change per pulse for the selected axis under real time input in the closed-loop mode.
only used in closed loop operation

Function specific parameters		
In	axis	[0 1 2]
	delta	to be added to current position in nm
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtin.setRealTimeInChangePerPulse		
params: [axis, delta]		
Result: [errNo]		
C-DLL call		
int AMC_rtin_setRealTimeInChangePerPulse (int deviceHandle, int axis, int delta)		
Python		
[dev].rtin.setRealTimeInChangePerPulse (axis, delta)		
Matlab		
[] = rtin_setRealTimeInChangePerPulse (axis, delta)		
C#		
void value = [Device]. Rtin_SetRealTimeInChangePerPulse (int axis, int delta)		
LabVIEW		
rtin_setRealTimeInChangePerPulse.vi		



setRealTimeInFeedbackLoopMode

Set if the realtime function must operate in close loop operation or open loop operation

Function specific parameters		
In	axis	[0 1 2]
	mode	0: open loop, 1 : close-loop
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtin.setRealTimeInFeedbackLoopMode		
params: [axis, mode]		
Result: [errNo]		
C-DLL call		
int AMC_rtin_setRealTimeInFeedbackLoopMode (int deviceHandle, int axis, int mode)		
Python		
[dev].rtin.setRealTimeInFeedbackLoopMode(axis, mode)		
Matlab		
[] = rtin_setRealTimeInFeedbackLoopMode(axis, mode)		
C#		
void value = [Device].Rtin_SetRealTimeInFeedbackLoopMode(int axis, int mode)		
LabVIEW		
rtin_setRealTimeInFeedbackLoopMode.vi		

setRealTimeInMode

This function sets the real time input mode for the selected axis.

Function specific parameters		
In	axis	[0 1 2]
	RT_IN_MODE	0: AquadB (LVTTTL) 1: AquadB (LVDS) 8: Stepper (LVTTTL) 9: Stepper (LVDS) 10: Trigger (LVTTTL) 11: Trigger (LVDS) 15: Off
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtin.setRealTimeInMode		
params: [axis, RT_IN_MODE]		
Result: [errNo]		
C-DLL call		
int AMC_rtin_setRealTimeInMode (int deviceHandle, int axis, int RT_IN_MODE)		
Python		
[dev].rtin.setRealTimeInMode (axis, RT_IN_MODE)		
Matlab		
[] = rtin_setRealTimeInMode (axis, RT_IN_MODE)		
C#		
void value = [Device]. Rtin_SetRealTimeInMode (int axis, int RT_IN_MODE)		
LabVIEW		
rtin_setRealTimeInMode.vi		



setRealTimeInStepsPerPulse

Set the change in step per pulse of the realtime input when trigger and stepper mode is used
only used in open loop operation

Function specific parameters		
In	axis	[0 1 2]
	steps	number of steps to applied
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtin.setRealTimeInStepsPerPulse		
params: [axis, steps]		
Result: [errNo]		
C-DLL call		
int AMC_rtin_setRealTimeInStepsPerPulse (int deviceHandle, int axis, int steps)		
Python		
[dev].rtin.setRealTimeInStepsPerPulse (axis, steps)		
Matlab		
[] = rtin_setRealTimeInStepsPerPulse (axis, steps)		
C#		
void value = [Device]. Rtin_SetRealTimeInStepsPerPulse (int axis, int steps)		
LabVIEW		
rtin_setRealTimeInStepsPerPulse.vi		

3.12. Rtout

apply

Apply for all rtout function

Function specific parameters		
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtout.apply		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_rtout_apply (int deviceHandle)		
Python		
[dev].rtout.apply()		
Matlab		
[] = rtout_apply()		
C#		
void value = [Device]. Rtout_Apply ()		
LabVIEW		
rtout_apply.vi		



applyAxis

Apply for rtout function of specific axis

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtout.applyAxis		
params: [axis]		
Result: [errNo]		
C-DLL call		
int AMC_rtout_applyAxis (int deviceHandle, int axis)		
Python		
[dev].rtout.applyAxis(axis)		
Matlab		
[] = rtout_applyAxis(axis)		
C#		
void value = [Device].Rtout_ApplyAxis(int axis)		
LabVIEW		
rtout_applyAxis.vi		

discard

Discard all rtout value set by the set function(not applied yet)

Function specific parameters		
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtout.discard		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_rtout_discard (int deviceHandle)		
Python		
[dev].rtout.discard()		
Matlab		
[] = rtout_discard()		
C#		
void value = [Device].Rtout_Discard()		
LabVIEW		
rtout_discard.vi		



discardAxis

Discard rtout value of specific axis set by the set function(not applied yet)

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtout.discardAxis		
params: [axis]		
Result: [errNo]		
C-DLL call		
int AMC_rtout_discardAxis (int deviceHandle, int axis)		
Python		
[dev].rtout.discardAxis (axis)		
Matlab		
[] = rtout_discardAxis (axis)		
C#		
void value = [Device]. Rtout_DiscardAxis (int axis)		
LabVIEW		
rtout_discardAxis.vi		

discardSignalMode

Discard value set by setSignalMode

Function specific parameters		
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtout.discardSignalMode		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_rtout_discardSignalMode (int deviceHandle)		
Python		
[dev].rtout.discardSignalMode()		
Matlab		
[] = rtout_discardSignalMode()		
C#		
void value = [Device].Rtout_DiscardSignalMode()		
LabVIEW		
rtout_discardSignalMode.vi		



getControlAQuadBOut

This function gets if of AQuadB output for position indication is enabled

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	enabled	boolean
JSON Method		
method: com.attocube.amc.rtout.getControlAQuadBOut		
params: [axis]		
Result: [errNo, enabled]		
C-DLL call		
int AMC_rtout_getControlAQuadBOut (int deviceHandle, int axis, bool* enabled)		
Python		
enabled = [dev].rtout.getControlAQuadBOut(axis)		
Matlab		
[enabled] = rtout_getControlAQuadBOut (axis)		
C#		
bool value = [Device]. Rtout_GetControlAQuadBOut (int axis)		
LabVIEW		
rtout_getControlAQuadBOut.vi		

getControlAQuadBOutClock

This function gets the clock for AQuadB output.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	clock_in_ns	Clock in multiples of 20ns. Minimum 2 (40ns), maximum 65535 (1,310700ms)
JSON Method		
method: com.attocube.amc.rtout.getControlAQuadBOutClock		
params: [axis]		
Result: [errNo, clock_in_ns]		
C-DLL call		
int AMC_rtout_getControlAQuadBOutClock (int deviceHandle, int axis, int* clock_in_ns)		
Python		
clock_in_ns = [dev].rtout.getControlAQuadBOutClock(axis)		
Matlab		
[clock_in_ns] = rtout_getControlAQuadBOutClock(axis)		
C#		
int value = [Device].Rtout_GetControlAQuadBOutClock(int axis)		
LabVIEW		
rtout_getControlAQuadBOutClock.vi		



getControlAQuadBOutResolution

This function gets the AQuadB output resolution for position indication.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	resolution	in nm
JSON Method		
method: com.attocube.amc.rtout.getControlAQuadBOutResolution		
params: [axis]		
Result: [errNo, resolution]		
C-DLL call		
int AMC_rtout_getControlAQuadBOutResolution (int deviceHandle, int axis, int* resolution)		
Python		
resolution = [dev].rtout.getControlAQuadBOutResolution(axis)		
Matlab		
[resolution] = rtout_getControlAQuadBOutResolution (axis)		
C#		
int value = [Device]. Rtout_GetControlAQuadBOutResolution (int axis)		
LabVIEW		
rtout_getControlAQuadBOutResolution.vi		

getMode

Get Mode

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	RT_OUT_MODE	For the meaning of the values, please refer to the setMode-function
JSON Method		
method: com.attocube.amc.rtout.getMode		
params: [axis]		
Result: [errNo, RT_OUT_MODE]		
C-DLL call		
int AMC_rtout_getMode (int deviceHandle, int axis, int* RT_OUT_MODE)		
Python		
RT_OUT_MODE = [dev].rtout.getMode(axis)		
Matlab		
[RT_OUT_MODE] = rtout_getMode(axis)		
C#		
int value = [Device].Rtout_GetMode(int axis)		
LabVIEW		
rtout_getMode.vi		



getSignalMode

This function gets the real time output mode for the selected axis.

Function specific parameters		
Out	errNo	errNo
	mode	0: TTL, 1: LVDS
JSON Method		
method: com.attocube.amc.rtout.getSignalMode		
params: []		
Result: [errNo, mode]		
C-DLL call		
int AMC_rtout_getSignalMode (int deviceHandle, int* mode)		
Python		
mode = [dev].rtout.getSignalMode()		
Matlab		
[mode] = rtout_getSignalMode()		
C#		
int value = [Device].Rtout_GetSignalMode()		
LabVIEW		
rtout_getSignalMode.vi		

getTriggerConfig

Get the real time output trigger config

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	higher	upper limit in nm / μ deg
	lower	lower limit in nm / μ deg
	epsilon	hysteresis in nm / μ deg
	polarity	0: active high, 1: active low
JSON Method		
method: com.attocube.amc.rtout.getTriggerConfig		
params: [axis]		
Result: [errNo, higher, lower, epsilon, polarity]		
C-DLL call		
int AMC_rtout_getTriggerConfig (int deviceHandle, int axis, int* higher, int* lower, int* epsilon, int* polarity)		
Python		
higher, lower, epsilon, polarity = [dev].rtout.getTriggerConfig(axis)		
Matlab		
[higher, lower, epsilon, polarity] = rtout_getTriggerConfig(axis)		
C#		
Tuple<int,int,int,int> value = [Device].Rtout_GetTriggerConfig(int axis)		
LabVIEW		
rtout_getTriggerConfig.vi		



setControlAQuadBOutClock

This function sets the clock for AQuadB output.

Function specific parameters		
In	axis	[0 1 2]
	clock	Clock in multiples of 20ns. Minimum 2 (40ns), maximum 65535 (1,310700ms)
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtout.setControlAQuadBOutClock		
params: [axis, clock]		
Result: [errNo]		
C-DLL call		
int AMC_rtout_setControlAQuadBOutClock (int deviceHandle, int axis, int clock)		
Python		
[dev].rtout.setControlAQuadBOutClock(axis, clock)		
Matlab		
[] = rtout_setControlAQuadBOutClock(axis, clock)		
C#		
void value = [Device].Rtout_SetControlAQuadBOutClock(int axis, int clock)		
LabVIEW		
rtout_setControlAQuadBOutClock.vi		

setControlAQuadBOutResolution

This function sets the AQuadB output resolution for position indication.

Function specific parameters		
In	axis	[0 1 2]
	resolution	in nm; range [1 ... 16777]
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtout.setControlAQuadBOutResolution		
params: [axis, resolution]		
Result: [errNo]		
C-DLL call		
int AMC_rtout_setControlAQuadBOutResolution (int deviceHandle, int axis, int resolution)		
Python		
[dev].rtout.setControlAQuadBOutResolution (axis, resolution)		
Matlab		
[] = rtout_setControlAQuadBOutResolution (axis, resolution)		
C#		
void value = [Device]. Rtout_SetControlAQuadBOutResolution (int axis, int resolution)		
LabVIEW		
rtout_setControlAQuadBOutResolution.vi		



setMode

Set the real time output signal mode. To set either LVTTTL or LVDS, please use setSignalMode-function

Function specific parameters		
In	axis	[0 1 2]
	RT_OUT_MODE	0: Off, 1: AquadB, 2: Trigger
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtout.setMode		
params: [axis, RT_OUT_MODE]		
Result: [errNo]		
C-DLL call		
int AMC_rtout_setMode (int deviceHandle, int axis, int RT_OUT_MODE)		
Python		
[dev].rtout.setMode(axis, RT_OUT_MODE)		
Matlab		
[] = rtout_setMode(axis, RT_OUT_MODE)		
C#		
void value = [Device].Rtout_SetMode(int axis, int RT_OUT_MODE)		
LabVIEW		
rtout_setMode.vi		

setSignalMode

This function sets the real time output mode for the selected axis.

Function specific parameters		
In	mode	0: TTL, 1: LVDS
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtout.setSignalMode		
params: [mode]		
Result: [errNo]		
C-DLL call		
int AMC_rtout_setSignalMode (int deviceHandle, int mode)		
Python		
[dev].rtout.setSignalMode(mode)		
Matlab		
[] = rtout_setSignalMode(mode)		
C#		
void value = [Device]. Rtout_SetSignalMode (int mode)		
LabVIEW		
rtout_setSignalMode.vi		



setTriggerConfig

Control the real time output trigger config

Function specific parameters		
In	axis	[0 1 2]
	higher	upper limit in nm / μ deg
	lower	lower limit in nm / μ deg
	epsilon	hysteresis in nm / μ deg
	polarity	0: active high, 1: active low
Out	errNo	errNo
JSON Method		
method: com.attocube.amc.rtout.setTriggerConfig		
params: [axis, higher, lower, epsilon, polarity]		
Result: [errNo]		
C-DLL call		
int AMC_rtout_setTriggerConfig (int deviceHandle, int axis, int higher, int lower, int epsilon, int polarity)		
Python		
[dev].rtout.setTriggerConfig (axis, higher, lower, epsilon, polarity)		
Matlab		
[] = rtout_setTriggerConfig (axis, higher, lower, epsilon, polarity)		
C#		
void value = [Device]. Rtout_SetTriggerConfig (int axis, int higher, int lower, int epsilon, int polarity)		
LabVIEW		
rtout_setTriggerConfig.vi		

3.13. Status

getFullCombinedStatus

Get the full combined status of a positioner axis and returns the status as a string

The following strings are possible:

Status	How to interpret	Comments
positioner not connected	No positioner connected	
output not enabled	Positioner connected, but axis deactivated	
ready	Axis active, not in closed loop, but ready to move	
moving	Axis moving (closed loop or open loop)	
in target range	Axis in closed loop and in target range	
backward limit reached	Axis backward limit reached (slower then velocity threshold) in backward direction	If deactivate on EOT is active the closed loop got stopped
forward limit reached	Axis backward limit reached (slower then velocity threshold) in forward direction	If deactivate on EOT is active the closed loop got stopped
grounded	Positioner grounded	Only AMC300
overtemperature	On the axis occurred an overtemperature	Only from next firmware 1.5.0, before the axis can be activated again, the temperature needs to fall below a certain threshold of 40°C.
output error	On the axis occurred an shortcut, the axis can not be activated	Only from next firmware 1.5.0, if an shortcut was detected another function needs to be called to delete the error. resetOutputHealthStatus()

Function specific parameters

In	axis	[0 1 2]
Out	errNo	errNo
	status_str	status_str

JSON Method

method: com.attocube.amc.status.getFullCombinedStatus

params: [axis]

Result: [errNo, status_str]

C-DLL call

int **AMC_status_getFullCombinedStatus**(int deviceHandle, int axis, char* status_str, int size0)

Python

status_str = [dev].status.getFullCombinedStatus(axis)

Matlab

[status_str] = status_getFullCombinedStatus(axis)

C#

string value = [Device].Status_GetFullCombinedStatus(int axis)

LabVIEW

getFullCombinedStatus.vi



getOIStatus

Get the Feedback status of the positioner

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	sensorstatus	as integer 0: NUM Positioner connected 1: OL positioner connected 2: No positioner connected , 3: RES positione connected, 4: Positioner with IDS-CL connected
JSON Method		
method: com.attocube.amc.status.getOIStatus		
params: [axis]		
Result: [errNo, sensorstatus]		
C-DLL call		
int AMC_status_getOIStatus (int deviceHandle, int axis, int* sensorstatus)		
Python		
sensorstatus = [dev].status.getOIStatus(axis)		
Matlab		
[sensorstatus] = status_getOIStatus (axis)		
C#		
int value = [Device]. Status_GetOIStatus (int axis)		
LabVIEW		
getOIStatus.vi		

getStatusConnected

This function gets information about the connection status of the selected axis' positioner.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	connected	If true, the actor is connected
JSON Method		
method: com.attocube.amc.status.getStatusConnected		
params: [axis]		
Result: [errNo, connected]		
C-DLL call		
int AMC_status_getStatusConnected (int deviceHandle, int axis, bool* connected)		
Python		
connected = [dev].status.getStatusConnected (axis)		
Matlab		
[connected] = status_getStatusConnected (axis)		
C#		
bool value = [Device]. Status_GetStatusConnected (int axis)		
LabVIEW		
getStatusConnected.vi		



getStatusEot

Retrieves the status of the end of travel (EOT) detection in backward direction or in forward direction.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	detected	true when EoT in either direction was detected
JSON Method		
method: com.attocube.amc.status.getStatusEot		
params: [axis]		
Result: [errNo, detected]		
C-DLL call		
int AMC_status_getStatusEot (int deviceHandle, int axis, bool* detected)		
Python		
detected = [dev].status.getStatusEot(axis)		
Matlab		
[detected] = status_getStatusEot (axis)		
C#		
bool value = [Device]. Status_GetStatusEot (int axis)		
LabVIEW		
getStatusEot.vi		

getStatusEotBkwd

This function gets the status of the end of travel detection on the selected axis in backward direction.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	detected	true when EoT was detected
JSON Method		
method: com.attocube.amc.status.getStatusEotBkwd		
params: [axis]		
Result: [errNo, detected]		
C-DLL call		
int AMC_status_getStatusEotBkwd (int deviceHandle, int axis, bool* detected)		
Python		
detected = [dev].status.getStatusEotBkwd(axis)		
Matlab		
[detected] = status_getStatusEotBkwd (axis)		
C#		
bool value = [Device]. Status_GetStatusEotBkwd (int axis)		
LabVIEW		
getStatusEotBkwd.vi		



getStatusEotFwd

This function gets the status of the end of travel detection on the selected axis in forward direction.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	detected	true when EoT was detected
JSON Method		
method: com.attocube.amc.status.getStatusEotFwd		
params: [axis]		
Result: [errNo, detected]		
C-DLL call		
int AMC_status_getStatusEotFwd (int deviceHandle, int axis, bool* detected)		
Python		
detected = [dev].status.getStatusEotFwd(axis)		
Matlab		
[detected] = status_getStatusEotFwd (axis)		
C#		
bool value = [Device]. Status_GetStatusEotFwd (int axis)		
LabVIEW		
getStatusEotFwd.vi		

getStatusFlagsAllAxesJson

This function returns several status flags from all axes as json-string.

The json-string has the following structure (please note, that json does not guarantee the order of the entries, i.e. axis_1 may be the second or last entry in the function return value):

```
{
  "axis_1": {
    "<name of status-flag>": true|false,
    ...
  },
  "axis_2": {
    "<name of status-flag>": true|false,
    ...
  },
  "axis_3": {
    "<name of status-flag>": true|false,
    ...
  }
}
```

The following status flags are included in the json-string (please note, that json does not guarantee the order of the entries, i.e. "actuator_connected" may be at a different position within the "axis_X"-field):

- actuator_connected: indicates whether a positioner is connected to the axis (for more information please refer to getStatusConnected)
- axis_active: indicates whether the axis is active (enabled) (for more information please refer to getControlOutput)
- moving: indicates whether the axis is currently moving (for more information please refer to getStatusMoving)
- in_target_range: indicates whether the positioner is in target range (for more information please refer to getStatusTargetRange)
- eot_fwd: indicates whether the forward end of travel has been reached (for more information please refer to getStatusEotFwd)
- eot_bwd: indicates whether the backward end of travel has been reached (for more information please refer to getStatusEotBkwd)
- axis_grounded: indicates whether the axis is grounded (e.g. ground on target) (for more information please refer to getGroundAxis)
- short_detected: indicates whether a Short circuit has been detected on the axis (for more information please refer to getOutputHealthStatus)

Function specific parameters		
Out	err	err
	statusFlagsJson	json-string containing the status flags of all axes (please refer to the interface manual for more information)
JSON Method		
method: com.attocube.amc.status.getStatusFlagsAllAxesJson		
params: []		
Result: [err, statusFlagsJson]		
C-DLL call		
int AMC_status_getStatusFlagsAllAxesJson (int deviceHandle, int* err, char* statusFlagsJson, int size0)		
Python		
err, statusFlagsJson = [dev].status.getStatusFlagsAllAxesJson()		
Matlab		
[err, statusFlagsJson] = status_getStatusFlagsAllAxesJson()		
C#		
Tuple<int,string> value = [Device].Status_GetStatusFlagsAllAxesJson()		



LabVIEW
getStatusFlagsAllAxesJson.vi

getStatusFlagsAllAxis

This function returns several status flags of all three axes

Function specific parameters		
Out	err	err
	pos_con_ax1	positioner connected axis 1
	enbld_ax1	Axis 1 enabled
	moving_ax1	axis 1 moving
	in_trgt_ax1	axis 1 in target range
	eot_fwd_ax1	axis 1 eot fwd flag
	eot_bwd_ax1	axis 1 eot bwd flag
	gnd_ax1	axis 1 grounded (e.g. ground on target)
	pos_con_ax2	positioner connected axis 2
	enbld_ax2	Axis 2 enabled
	moving_ax2	axis 2 moving
	in_trgt_ax2	axis 2 in target range
	eot_fwd_ax2	axis 2 eot fwd flag
	eot_bwd_ax2	axis 2 eot bwd flag
	gnd_ax2	axis 2 grounded (e.g. ground on target)
	pos_con_ax3	positioner connected axis 3
	enbld_ax3	Axis 3 enabled
	moving_ax3	axis 3 moving
	in_trgt_ax3	axis 3 in target range
	eot_fwd_ax3	axis 3 eot fwd flag
	eot_bwd_ax3	axis 3 eot bwd flag
	gnd_ax3	axis 3 grounded (e.g. ground on target)
JSON Method		
method: com.attocube.amc.status.getStatusFlagsAllAxis		
params: []		
Result: [err, pos_con_ax1, enbld_ax1, moving_ax1, in_trgt_ax1, eot_fwd_ax1, eot_bwd_ax1, gnd_ax1, pos_con_ax2, enbld_ax2, moving_ax2, in_trgt_ax2, eot_fwd_ax2, eot_bwd_ax2, gnd_ax2, pos_con_ax3, enbld_ax3, moving_ax3, in_trgt_ax3, eot_fwd_ax3, eot_bwd_ax3, gnd_ax3]		
C-DLL call		
int AMC_status_getStatusFlagsAllAxis (int deviceHandle, int* err, bool* pos_con_ax1, bool* enbld_ax1, bool* moving_ax1, bool* in_trgt_ax1, bool* eot_fwd_ax1, bool* eot_bwd_ax1, bool* gnd_ax1, bool* pos_con_ax2, bool* enbld_ax2, bool* moving_ax2, bool* in_trgt_ax2, bool* eot_fwd_ax2, bool* eot_bwd_ax2, bool* gnd_ax2, bool* pos_con_ax3, bool* enbld_ax3, bool* moving_ax3, bool* in_trgt_ax3, bool* eot_fwd_ax3, bool* eot_bwd_ax3, bool* gnd_ax3)		
Python		
err, pos_con_ax1, enbld_ax1, moving_ax1, in_trgt_ax1, eot_fwd_ax1, eot_bwd_ax1, gnd_ax1, pos_con_ax2, enbld_ax2, moving_ax2, in_trgt_ax2, eot_fwd_ax2, eot_bwd_ax2, gnd_ax2, pos_con_ax3, enbld_ax3, moving_ax3, in_trgt_ax3, eot_fwd_ax3, eot_bwd_ax3, gnd_ax3 = [dev].status.getStatusFlagsAllAxis()		
Matlab		
[err, pos_con_ax1, enbld_ax1, moving_ax1, in_trgt_ax1, eot_fwd_ax1, eot_bwd_ax1, gnd_ax1, pos_con_ax2, enbld_ax2, moving_ax2, in_trgt_ax2, eot_fwd_ax2, eot_bwd_ax2, gnd_ax2, pos_con_ax3, enbld_ax3, moving_ax3, in_trgt_ax3, eot_fwd_ax3, eot_bwd_ax3, gnd_ax3] = status_getStatusFlagsAllAxis()		
C#		



Tuple<int,bool,bool,bool,bool,bool, Tuple<bool,bool,bool,bool,bool,bool,bool, Tuple<bool,bool,bool,bool,bool,bool, Tuple<bool,bool,bool>>>> value = [Device]. Status_GetStatusFlagsAllAxis()
--

LabVIEW

getStatusFlagsAllAxis.vi

getStatusMoving

This function gets information about the status of the stage output.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	status	0: Idle, i.e. within the noise range of the sensor, 1: Moving, i.e the actor is actively driven by the output stage either for closed-loop approach or continuous/single stepping and the output is active. 2 : Pending means the output stage is driving but the output is deactivated
JSON Method		
method: com.attocube.amc.status.getStatusMoving		
params: [axis]		
Result: [errNo, status]		
C-DLL call		
int AMC_status_getStatusMoving (int deviceHandle, int axis, int* status)		
Python		
status = [dev].status.getStatusMoving(axis)		
Matlab		
[status] = status_getStatusMoving (axis)		
C#		
int value = [Device]. Status_GetStatusMoving (int axis)		
LabVIEW		
getStatusMoving.vi		



getStatusReference

This function gets information about the status of the reference position.

It can only be used in conjunction with NUM.

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	valid	true = valid, false = not valid
JSON Method		
method: com.attocube.amc.status.getStatusReference		
params: [axis]		
Result: [errNo, valid]		
C-DLL call		
int AMC_status_getStatusReference (int deviceHandle, int axis, bool* valid)		
Python		
valid = [dev].status.getStatusReference(axis)		
Matlab		
[valid] = status_getStatusReference (axis)		
C#		
bool value = [Device]. Status_GetStatusReference (int axis)		
LabVIEW		
getStatusReference.vi		

getStatusTargetRange

This function gets information about whether the selected axis' positioner is in target range or not.

The detection only indicates whether the position is within the defined range. This status is updated periodically but currently not in real-time.

If a fast detection is desired, please check the position in a loop

Function specific parameters		
In	axis	[0 1 2]
Out	errNo	errNo
	in_range	true within the target range, false not within the target range
JSON Method		
method: com.attocube.amc.status.getStatusTargetRange		
params: [axis]		
Result: [errNo, in_range]		
C-DLL call		
int AMC_status_getStatusTargetRange (int deviceHandle, int axis, bool* in_range)		
Python		
in_range = [dev].status.getStatusTargetRange(axis)		
Matlab		
[in_range] = status_getStatusTargetRange (axis)		
C#		
bool value = [Device]. Status_GetStatusTargetRange (int axis)		
LabVIEW		
getStatusTargetRange.vi		



3.14. System_service

errorNumberToRecommendation

Gets a recommendation for the error code.

Function specific parameters		
In	language	integer: Language code
	errNbr	error code to translate
Out	errNo	error code, if there was an error, otherwise 0 for ok
	value_string1	string: error recommendation (currently returning an int = 0 until we have recommendations)
JSON Method		
method: com.attocube.system.errorNumberToRecommendation		
params: [language, errNbr]		
Result: [errNo, value_string1]		
C-DLL call		
int AMC_errorNumberToRecommendation (int deviceHandle, int language, int errNbr, char* value_string1, int size0)		
Python		
value_string1 = [dev].system_service.errorNumberToRecommendation(language, errNbr)		
Matlab		
[value_string1] = AMC_errorNumberToRecommendation (language, errNbr)		
C#		
string value = [Device]. ErrorNumberToRecommendation (int language, int errNbr)		
LabVIEW		
errorNumberToRecommendation.vi		

errorNumberToString

Gets a description of an error code.

Function specific parameters		
In	language	integer: Language code 0 for the error name, 1 for a more user-friendly error message
	errNbr	error code to translate
Out	errNo	error code, if there was an error, otherwise 0 for ok
	message	error description
JSON Method		
method: com.attocube.system.errorNumberToString		
params: [language, errNbr]		
Result: [errNo, message]		
C-DLL call		
int AMC_errorNumberToString (int deviceHandle, int language, int errNbr, char* message, int size0)		
Python		
message = [dev].system_service.errorNumberToString(language, errNbr)		
Matlab		
[message] = AMC_errorNumberToString (language, errNbr)		
C#		
string value = [Device]. ErrorNumberToString (int language, int errNbr)		
LabVIEW		
errorNumberToString.vi		



factoryReset

Turns on the factory reset flag. To perform the factory reset, a reboot is necessary afterwards. All settings will be set to default, and the device will be configured as DHCP server.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.factoryReset		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_factoryReset (int deviceHandle)		
Python		
[dev].system_service.factoryReset()		
Matlab		
[] = AMC_factoryReset ()		
C#		
void value = [Device]. FactoryReset ()		
LabVIEW		
factoryReset.vi		

getDeviceName

Gets the current device name.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	name	current device name
JSON Method		
method: com.attocube.system.getDeviceName		
params: []		
Result: [errNo, name]		
C-DLL call		
int AMC_getDeviceName (int deviceHandle, char* name, int size0)		
Python		
name = [dev].system_service.getDeviceName()		
Matlab		
[name] = AMC_getDeviceName ()		
C#		
string value = [Device]. GetDeviceName ()		
LabVIEW		
getDeviceName.vi		



getFirmwareVersion

Gets the firmware version of the device.

Function specific parameters		
Out	errNo	error code to translate
	FWversion	firmware version
JSON Method		
method: com.attocube.system.getFirmwareVersion		
params: []		
Result: [errNo, FWversion]		
C-DLL call		
int AMC_getFirmwareVersion (int deviceHandle, char* FWversion, int size0)		
Python		
FWversion = [dev].system_service.getFirmwareVersion()		
Matlab		
[FWversion] = AMC_getFirmwareVersion ()		
C#		
string value = [Device]. GetFirmwareVersion ()		
LabVIEW		
getFirmwareVersion.vi		

getFluxCode

Gets the flux code of the system.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	code	flux code
JSON Method		
method: com.attocube.system.getFluxCode		
params: []		
Result: [errNo, code]		
C-DLL call		
int AMC_getFluxCode (int deviceHandle, char* code, int size0)		
Python		
code = [dev].system_service.getFluxCode()		
Matlab		
[code] = AMC_getFluxCode ()		
C#		
string value = [Device]. GetFluxCode ()		
LabVIEW		
getFluxCode.vi		



getHardwareInfo

Retrieves the hardware information of the device.

Function specific parameters		
Out	errNo	error code
	hardware_info_string	hardware_info_string
JSON Method		
method: com.attocube.system.getHardwareInfo		
params: []		
Result: [errNo, hardware_info_string]		
C-DLL call		
int AMC_getHardwareInfo (int deviceHandle, char* hardware_info_string, int size0)		
Python		
hardware_info_string = [dev].system_service.getHardwareInfo()		
Matlab		
[hardware_info_string] = AMC_getHardwareInfo ()		
C#		
string value = [Device]. GetHardwareInfo ()		
LabVIEW		
getHardwareInfo.vi		

getHostname

Returns the device hostname.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	name	hostname
JSON Method		
method: com.attocube.system.getHostname		
params: []		
Result: [errNo, name]		
C-DLL call		
int AMC_getHostname (int deviceHandle, char* name, int size0)		
Python		
name = [dev].system_service.getHostname()		
Matlab		
[name] = AMC_getHostname ()		
C#		
string value = [Device]. GetHostname ()		
LabVIEW		
getHostname.vi		



getMacAddress

Gets the mac address of the device.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	mac	mac address
JSON Method		
method: com.attocube.system.getMacAddress		
params: []		
Result: [errNo, mac]		
C-DLL call		
int AMC_getMacAddress (int deviceHandle, char* mac, int size0)		
Python		
mac = [dev].system_service.getMacAddress()		
Matlab		
[mac] = AMC_getMacAddress ()		
C#		
string value = [Device]. GetMacAddress ()		
LabVIEW		
getMacAddress.vi		

getSerialNumber

Gets the serial number of the device.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	SN	serial number
JSON Method		
method: com.attocube.system.getSerialNumber		
params: []		
Result: [errNo, SN]		
C-DLL call		
int AMC_getSerialNumber (int deviceHandle, char* SN, int size0)		
Python		
SN = [dev].system_service.getSerialNumber()		
Matlab		
[SN] = AMC_getSerialNumber ()		
C#		
string value = [Device]. GetSerialNumber ()		
LabVIEW		
getSerialNumber.vi		



rebootSystem

Reboots the device.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.rebootSystem		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_rebootSystem (int deviceHandle)		
Python		
[dev].system_service.rebootSystem()		
Matlab		
[] = AMC_rebootSystem ()		
C#		
void value = [Device]. RebootSystem ()		
LabVIEW		
rebootSystem.vi		

setDeviceName

Sets custom name for the device.

Function specific parameters		
In	name	device name
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.setDeviceName		
params: [name]		
Result: [errNo]		
C-DLL call		
int AMC_setDeviceName (int deviceHandle, const char* name)		
Python		
[dev].system_service.setDeviceName(name)		
Matlab		
[] = AMC_setDeviceName (name)		
C#		
void value = [Device]. SetDeviceName (string name)		
LabVIEW		
setDeviceName.vi		



setTime

Sets system time manually.

Function specific parameters		
In	day	value from 1 to 31
	month	value from 1 to 12
	year	value (e.g. 2021)
	hour	value from 0 to 23
	minute	value from 0 to 59
	second	value from 0 to 59
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.setTime		
params: [day, month, year, hour, minute, second]		
Result: [errNo]		
C-DLL call		
int AMC_setTime (int deviceHandle, int day, int month, int year, int hour, int minute, int second)		
Python		
[dev].system_service.setTime(day, month, year, hour, minute, second)		
Matlab		
[] = AMC_setTime (day, month, year, hour, minute, second)		
C#		
void value = [Device]. SetTime (int day, int month, int year, int hour, int minute, int second)		
LabVIEW		
setTime.vi		

softReset

Performs a soft reset (factory reset without deleting the network settings). Please reboot the device directly afterwards.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.softReset		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_softReset (int deviceHandle)		
Python		
[dev].system_service.softReset()		
Matlab		
[] = AMC_softReset ()		
C#		
void value = [Device]. SoftReset ()		
LabVIEW		
softReset.vi		



updateTimeFromInternet

Updates system time by querying attocube.com.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.updateTimeFromInternet		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_updateTimeFromInternet (int deviceHandle)		
Python		
[dev].system_service.updateTimeFromInternet()		
Matlab		
[] = AMC_updateTimeFromInternet ()		
C#		
void value = [Device]. UpdateTimeFromInternet ()		
LabVIEW		
updateTimeFromInternet.vi		

3.15. Update

getLicenseUpdateProgress

Gets the progress of running license update.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	progress	value in %
JSON Method		
method: com.attocube.system.update.getLicenseUpdateProgress		
params: []		
Result: [errNo, progress]		
C-DLL call		
int AMC_update_getLicenseUpdateProgress (int deviceHandle, int* progress)		
Python		
progress = [dev].update.getLicenseUpdateProgress()		
Matlab		
[progress] = update_getLicenseUpdateProgress ()		
C#		
int value = [Device]. Update_GetLicenseUpdateProgress ()		
LabVIEW		
getLicenseUpdateProgress.vi		



getSwUpdateProgress

Gets the progress of running update.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
	progress	value in %
JSON Method		
method: com.attocube.system.update.getSwUpdateProgress		
params: []		
Result: [errNo, progress]		
C-DLL call		
int AMC_update_getSwUpdateProgress (int deviceHandle, int* progress)		
Python		
progress = [dev].update.getSwUpdateProgress()		
Matlab		
[progress] = update_getSwUpdateProgress ()		
C#		
int value = [Device]. Update_GetSwUpdateProgress ()		
LabVIEW		
getSwUpdateProgress.vi		

licenseUpdateBase64

Executes the license update with base64 file uploaded. After execution, a manual reboot is necessary.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.update.licenseUpdateBase64		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_update_licenseUpdateBase64 (int deviceHandle)		
Python		
[dev].update.licenseUpdateBase64()		
Matlab		
[] = update_licenseUpdateBase64()		
C#		
void value = [Device].Update_LicenseUpdateBase64()		
LabVIEW		
licenseUpdateBase64.vi		



softwareUpdateBase64

Executes the update with base 64 file uploaded. After completion, a manual reboot is necessary.

Function specific parameters		
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.update.softwareUpdateBase64		
params: []		
Result: [errNo]		
C-DLL call		
int AMC_update_softwareUpdateBase64 (int deviceHandle)		
Python		
[dev].update.softwareUpdateBase64()		
Matlab		
[] = update_softwareUpdateBase64()		
C#		
void value = [Device]. Update_SoftwareUpdateBase64()		
LabVIEW		
softwareUpdateBase64.vi		

uploadLicenseBase64

Uploads new license file in format base 64.

Function specific parameters		
In	offset	offset of the data
	b64Data	base64 data string
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.update.uploadLicenseBase64		
params: [offset, b64Data]		
Result: [errNo]		
C-DLL call		
int AMC_update_uploadLicenseBase64 (int deviceHandle, int offset, const char* b64Data)		
Python		
[dev].update.uploadLicenseBase64 (offset, b64Data)		
Matlab		
[] = update_uploadLicenseBase64 (offset, b64Data)		
C#		
void value = [Device]. Update_UploadLicenseBase64 (int offset, string b64Data)		
LabVIEW		
uploadLicenseBase64.vi		



uploadSoftwareImageBase64

Uploads new firmware image in format base 64.

Function specific parameters		
In	offset	offset of the data
	b64Data	base64 data string
Out	errNo	error code, if there was an error, otherwise 0 for ok
JSON Method		
method: com.attocube.system.update.uploadSoftwareImageBase64		
params: [offset, b64Data]		
Result: [errNo]		
C-DLL call		
int AMC_update_uploadSoftwareImageBase64 (int deviceHandle, int offset, const char* b64Data)		
Python		
[dev].update.uploadSoftwareImageBase64 (offset, b64Data)		
Matlab		
[] = update_uploadSoftwareImageBase64 (offset, b64Data)		
C#		
void value = [Device]. Update_UploadSoftwareImageBase64 (int offset, string b64Data)		
LabVIEW		
uploadSoftwareImageBase64.vi		



attocube systems GmbH

Eglfinger Weg 2
85540 Haar, Germany
Phone: +49-89 -420 7970
E-Mail: info@attocube.com
www.attocube.com

For technical queries, contact:
support@attocube.com

Reginal US Sales Representatives:

West Coast: Arcadia, CA
Phone: +1 510 631 2154

Southwest: Henderson, NV
Phone: +1 510 631 1784

Midwest: Delaware, OH
Phone: +1 510 631 133



MOTION & SENSING
precision engineering components